

✓ Notebook Role: Phase 5 – Modeling Scaffold

This is the **core modeling notebook**:

- Loads Phase 3 artifacts (lexicon, hybrid, labels)
- Encodes labels consistently
- Provides an **editable model cell** for team hot-swaps
- Includes **GridSearchCV tuning** for linear models
- Hardened evaluation functions (accuracy, precision, recall, F1, ROC-AUC, confusion matrix, classification report)
- Runs Lexicon-onPly and Hybrid tracks with 5-fold CV
- Compares Macro-F1 scores visually

Purpose: **train/test with 5-fold CV** — reproducible, audit-ready evaluation pipeline for all team models.

✓ Section 1: Imports (Hardened)

Do not edit unless you need additional libraries.

This cell provides the common imports for all models and evaluations.

```
import pandas as pd
import numpy as np
from scipy import sparse
import joblib
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix, classification_report
)
from sklearn.model_selection import StratifiedKFold, cross_val_score
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Start coding or [generate](#) with AI.

```
import os
os.listdir('/content/drive/MyDrive/Colab Notebooks/Phase_3')
```

```
['chi2_selector.pkl',
'labels_train.csv',
'hybrid_features_metadata.json',
'labels_val.csv',
'tfidf_vectorizer.pkl',
'X_train_lexicon.npz',
'X_train_hybrid.npz',
'X_train_selected.npz',
'X_train_tfidf.npz',
'X_val_lexicon.npz',
'X_val_hybrid.npz',
'X_val_selected.npz',
'X_val_tfidf.npz',
'Phase_3.zip',
'svm_classification_report.csv',
'svm_results.xlsx']
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
from google.colab import files
```

```
from google.colab import files
```

```
df_report.to_csv("svm_results_final.csv")
files.download("svm_results_final.csv")
```

Section 2: Load Artifacts (Hardened)

Do not edit file paths unless Phase 3 outputs change.

This cell loads the lexicon-only and hybrid-weighted feature sets, plus labels.

Start coding or [generate](#) with AI.

```
X_train_lex = sparse.load_npz("/content/drive/MyDrive/Colab Notebooks/Phase_3/X_train_lexicon.npz").toarray()
X_val_lex   = sparse.load_npz("/content/drive/MyDrive/Colab Notebooks/Phase_3/X_val_lexicon.npz").toarray()

# Use the actual hybrid files you have
X_train_hybrid = sparse.load_npz("/content/drive/MyDrive/Colab Notebooks/Phase_3/X_train_hybrid.npz")
X_val_hybrid   = sparse.load_npz("/content/drive/MyDrive/Colab Notebooks/Phase_3/X_val_hybrid.npz")

y_train = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Phase_3/labels_train.csv")["Sentiment"]
y_val    = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Phase_3/labels_val.csv")["Sentiment"]

print("Lexicon train shape:", X_train_lex.shape)
print("Hybrid train shape:", X_train_hybrid.shape)
print("Train label distribution:\n", y_train.value_counts())
print("Val label distribution:\n", y_val.value_counts())
```

```
Lexicon train shape: (409577, 4)
Hybrid train shape: (409577, 3004)
Train label distribution:
Sentiment
positive    235384
negative    124820
neutral      49373
Name: count, dtype: int64
Val label distribution:
Sentiment
positive    6137739
negative     770638
neutral      578303
Name: count, dtype: int64
```

Start coding or [generate](#) with AI.

Section 3: Encode Labels (Hardened)

Do not edit.

This ensures consistent label encoding across all models.

```
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_val_enc   = le.transform(y_val)

print("Classes:", list(le.classes_))
```

```
Classes: ['negative', 'neutral', 'positive']
```

Start coding or [generate](#) with AI.

Section 4: Model Definition (Editable)

This is the **open cell**.

Each team member should define their own model here.

Examples:

- LogisticRegression(penalty='l2')
- LogisticRegression(penalty='l1', solver='saga')
- Perceptron()
- LinearSVC()
- RidgeClassifier()

- MLPClassifier(hidden_layer_sizes=(100,), activation='relu')

⚠ Only edit inside this cell. The rest of the notebook will use whatever `model` you define here.

Start coding or [generate](#) with AI.

```
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV

# Define your Linear SVM model wrapped with calibration
# LinearSVC is fast and scalable; calibration adds probabilities for ROC-AUC
base_model = LinearSVC(
    C=1.0,
    class_weight="balanced",
    random_state=42,
    max_iter=5000
)

model = CalibratedClassifierCV(
    estimator=base_model, # use 'estimator' instead of 'base_estimator'
    cv=3 # calibration folds
)

print("Section 4: LinearSVC + CalibratedClassifierCV model defined successfully")
print(model)
```

```
Section 4: LinearSVC + CalibratedClassifierCV model defined successfully
CalibratedClassifierCV(cv=3,
                      estimator=LinearSVC(class_weight='balanced',
                                           max_iter=5000, random_state=42))
```

Start coding or [generate](#) with AI.

Section 4a: Hyperparameter Tuning (Editable)

Do not edit.

This cell runs GridSearchCV on the model defined in Section 4, using `f1_macro` as the scoring metric.

It will search across a parameter grid appropriate for linear models (LogisticRegression, LinearSVC, RidgeClassifier, Perceptron).

Start coding or [generate](#) with AI.

```
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import GridSearchCV, train_test_split # Added imports

# Subsample training data for faster grid search
X_sub, _, y_sub, _ = train_test_split(
    X_train_lex, y_train_enc,
    train_size=50000, stratify=y_train_enc,
    random_state=42
)

# Parameter grid for LinearSVC (calibration wraps it, so tune base model)
param_grids = {
    "LinearSVC": {
        "C": [0.1, 1, 10],
        "loss": ["hinge", "squared_hinge"]
    }
}

# Detect model type and select grid
model_name = "LinearSVC"
param_grid = param_grids.get(model_name, {})

if param_grid:
    grid = GridSearchCV(
        estimator=LinearSVC(class_weight="balanced", random_state=42, max_iter=5000),
        param_grid=param_grid,
        scoring="f1_macro",
        cv=2, # fewer folds for speed
        n_jobs=-1,
        verbose=2
    )
    grid.fit(X_sub, y_sub)
    print(f"Best params for {model_name}:", grid.best_params_)
    print(f"Best CV Macro-F1 for {model_name}:", grid.best_score_)
```

```
# Refit final calibrated model on full training set with best params
tuned_base = LinearSVC(
    C=grid.best_params_["C"],
    loss=grid.best_params_["loss"],
    class_weight="balanced",
    random_state=42,
    max_iter=5000
)
model = CalibratedClassifierCV(
    estimator=tuned_base, # <-- use 'estimator' here
    cv=3
)
model.fit(X_train_lex, y_train_enc)
print("Final calibrated model retrained on full dataset with best params.")

else:
    print(f"No parameter grid defined for {model_name}. Please add one to param_grids.")
```

Fitting 2 folds for each of 6 candidates, totalling 12 fits
 Best params for LinearSVC: {'C': 0.1, 'loss': 'squared_hinge'}
 Best CV Macro-F1 for LinearSVC: 0.38534438602227694
 Final calibrated model retrained on full dataset with best params.

```
# PCA VISUALIZATION (Efficiency: Subsampled to 5000 points)
# =====
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

print("Generating PCA Visualization...")
plt.figure(figsize=(10, 7))

# Subsample for speed (PCA on 400k+ points is slow)
sample_size = 5000
indices = np.random.choice(X_val_hybrid.shape[0], sample_size, replace=False)

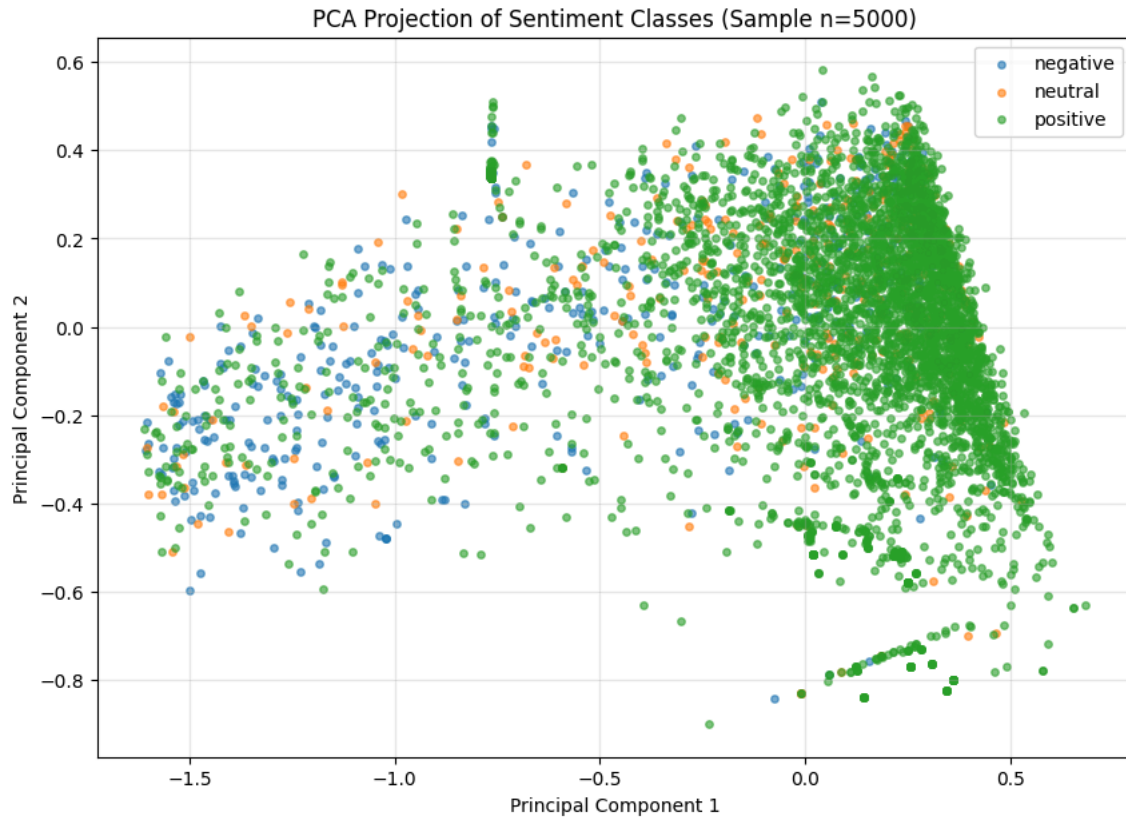
# Fix: Convert coo_matrix to csr_matrix for row-wise slicing
X_sample = X_val_hybrid.tocsr()[indices].toarray()
y_sample = y_val_enc[indices]

pca = PCA(n_components=2)
pca_results = pca.fit_transform(X_sample)

# Scatter plot for Positive, Neutral, Negative
for i, label in enumerate(le.classes_):
    mask = (y_sample == i)
    plt.scatter(pca_results[mask, 0], pca_results[mask, 1], label=label, alpha=0.6, s=15)

plt.title(f"PCA Projection of Sentiment Classes (Sample n={sample_size})")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Generating PCA Visualization...



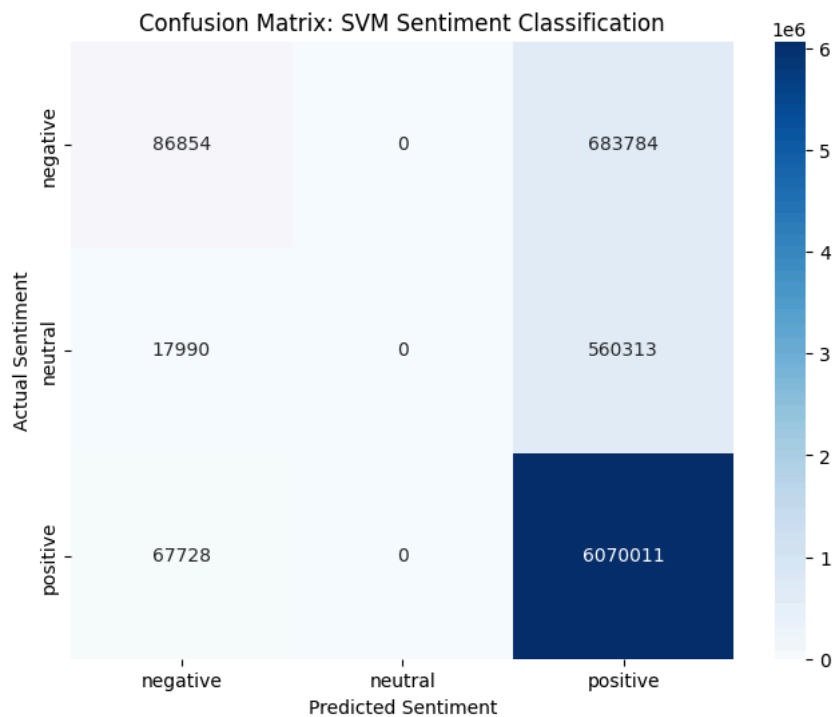
Confusion Matrix Graph

This shows exactly where the model is getting confused (e.g., misclassifying "Neutral" as "Positive").

```
def plot_confusion_matrix(y_true, y_pred, classes):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=classes, yticklabels=classes)
    plt.title('Confusion Matrix: SVM Sentiment Classification')
    plt.ylabel('Actual Sentiment')
    plt.xlabel('Predicted Sentiment')
    plt.show()

# Fix: Define y_pred and class_names
y_pred = model.predict(X_val_lex) # Make predictions on the validation set
class_names = le.classes_.tolist() # Get class names from the LabelEncoder

plot_confusion_matrix(y_val_enc, y_pred, class_names)
```



Start coding or [generate](#) with AI.

##Class Performance Comparison

This bar chart compares Precision, Recall, and F1-Score across Negative, Neutral, and Positive classes to identify which se

```
from sklearn.metrics import precision_recall_fscore_support

def plot_classification_metrics(y_true, y_pred, classes):
    precision, recall, f1, _ = precision_recall_fscore_support(y_true, y_pred)

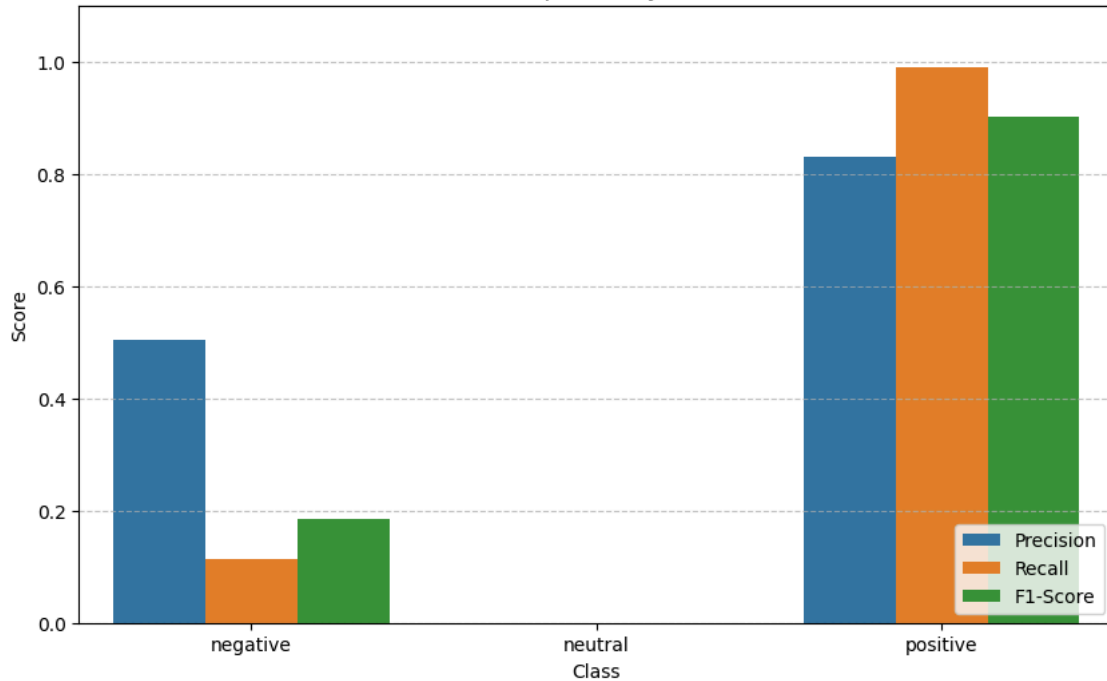
    metrics_df = pd.DataFrame({
        'Class': classes,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1
    }).melt(id_vars='Class', var_name='Metric', value_name='Score')

    plt.figure(figsize=(10, 6))
    sns.barplot(data=metrics_df, x='Class', y='Score', hue='Metric')
    plt.title('Performance Comparison by Sentiment Class')
    plt.ylim(0, 1.1)
    plt.legend(loc='lower right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

plot_classification_metrics(y_val_enc, y_pred, class_names)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Performance Comparison by Sentiment Class



#Multi-class ROC & Precision-Recall Curves

These curves are essential for understanding how the model handles the trade-off between sensitivity and specificity.

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve, average_precision_score
from sklearn.preprocessing import label_binarize
from itertools import cycle

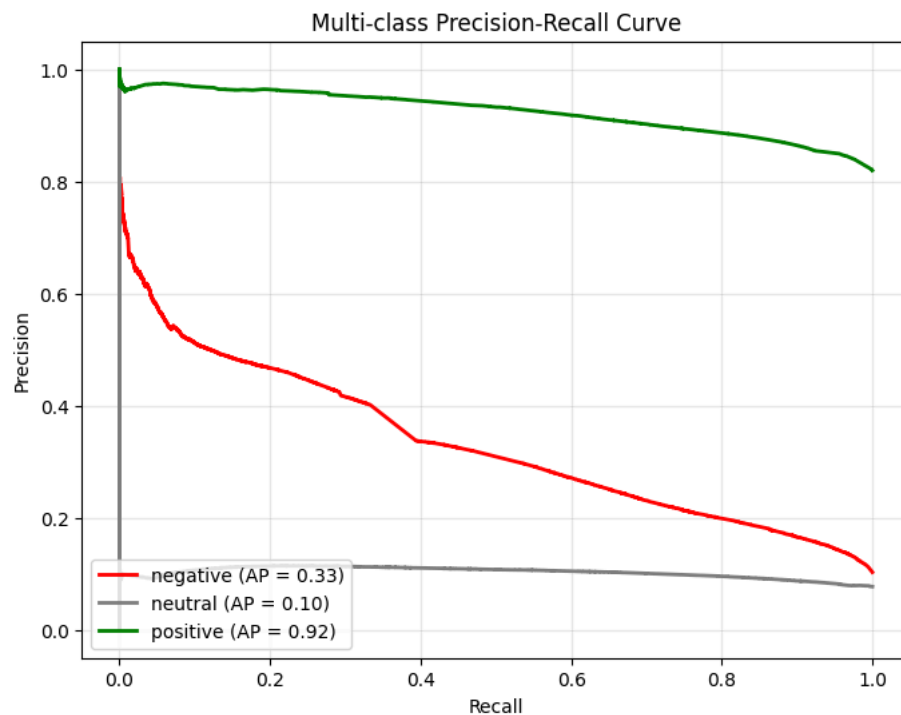
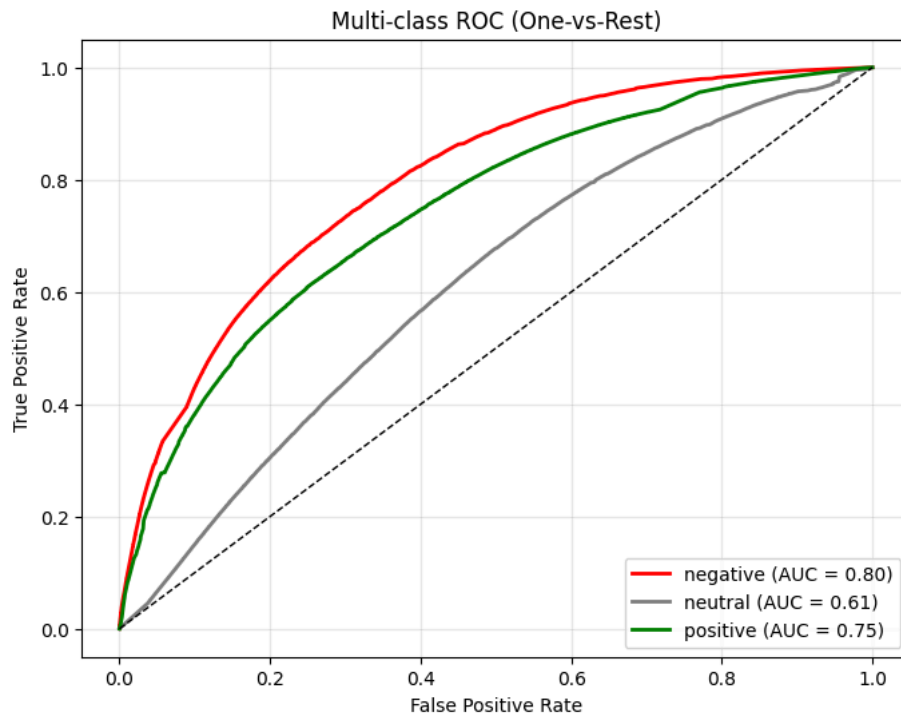
# Binarize the labels for multi-class plotting
y_val_bin = label_binarize(y_val_enc, classes=[0, 1, 2])
# Use predict_proba for CalibratedClassifierCV
y_score = model.predict_proba(X_val_lex)

# --- ROC Curve ---
plt.figure(figsize=(8, 6))
colors = cycle(['red', 'gray', 'green'])
for i, color in zip(range(len(class_names)), colors):
    fpr, tpr, _ = roc_curve(y_val_bin[:, i], y_score[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=color, lw=2, label=f'{class_names[i]} (AUC = {roc_auc:0.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=1)
plt.title('Multi-class ROC (One-vs-Rest)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()

# --- Precision-Recall Curve ---
plt.figure(figsize=(8, 6))
for i, color in zip(range(len(class_names)), colors):
    precision, recall, _ = precision_recall_curve(y_val_bin[:, i], y_score[:, i])
    ap = average_precision_score(y_val_bin[:, i], y_score[:, i])
    plt.plot(recall, precision, color=color, lw=2, label=f'{class_names[i]} (AP = {ap:0.2f})')

plt.title('Multi-class Precision-Recall Curve')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend(loc="lower left")
plt.grid(alpha=0.3)
plt.show()
```



##Actual vs. Predicted Distribution

This bar chart compares the frequency of your actual data labels versus what your SVM model predicted. If the "Predicted" t

##SVM Decision Boundaries (PCA Space)

This graph visualizes how the SVM actually "cuts" the data space to separate classes. We use PCA to reduce the lexicon feat

```
from sklearn.decomposition import PCA
from sklearn.svm import SVC # Import SVC

# 1. Reduce features to 2D
pca = PCA(n_components=2)
X_val_2d = pca.fit_transform(X_val_lex[:1000]) # Use subset for speed
y_subset = y_val_enc[:1000]

# 2. Fit a temporary SVM on 2D data to visualize the boundaries
svm_2d = SVC(kernel='rbf', C=1.0).fit(X_val_2d, y_subset)

# 3. Create Meshgrid
h = .05
x_min, x_max = X_val_2d[:, 0].min() - 1, X_val_2d[:, 0].max() + 1
y_min, y_max = X_val_2d[:, 1].min() - 1, X_val_2d[:, 1].max() + 1
```



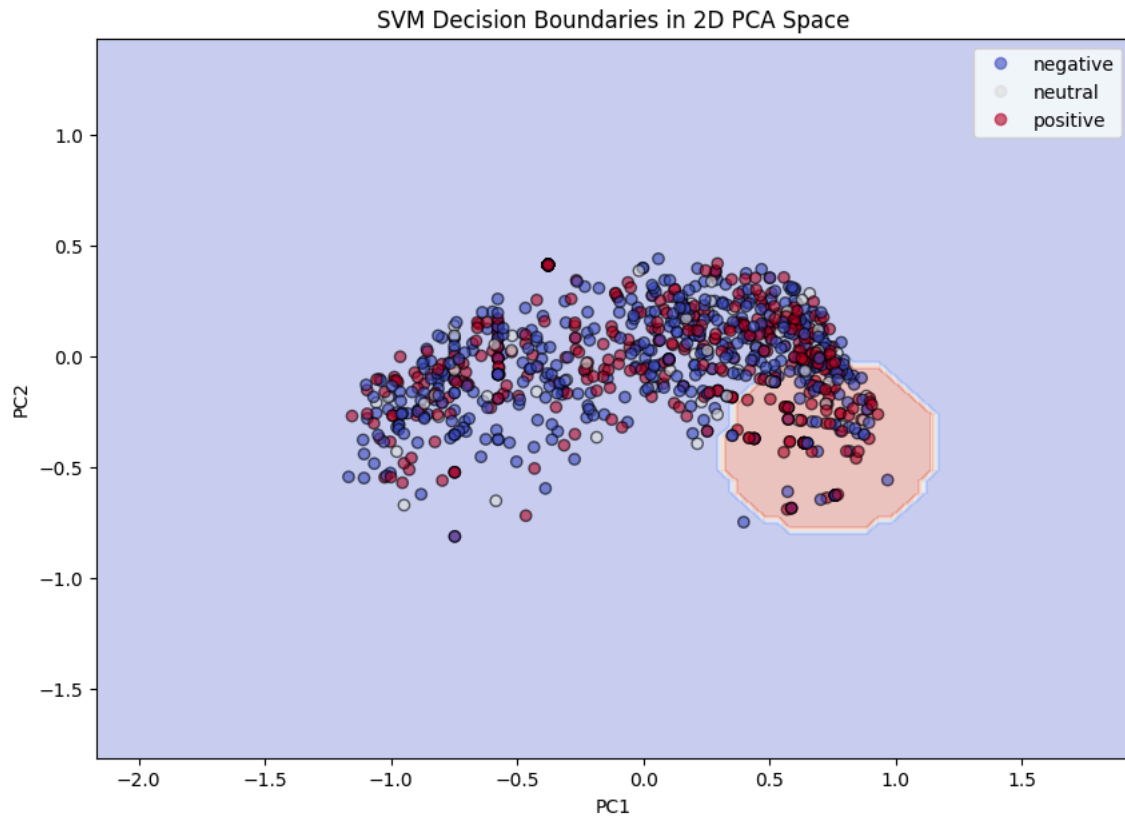
```

xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# 4. Predict over Meshgrid
Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(10, 7))
plt.contourf(xx, yy, Z, cmap='coolwarm', alpha=0.3)
scatter = plt.scatter(X_val_2d[:, 0], X_val_2d[:, 1], c=y_subset,
                      cmap='coolwarm', edgecolors='k', alpha=0.6)
plt.title('SVM Decision Boundaries in 2D PCA Space')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend(handles=scatter.legend_elements()[0], labels=list(class_names))
plt.show()

```



```

from sklearn.metrics import classification_report
import pandas as pd

# Make predictions using the defined model
y_pred = model.predict(X_val_lex)
class_names = le.classes_.tolist()

# 1. Generate the report as a dictionary
report_dict = classification_report(y_val_enc, y_pred, target_names=class_names, output_dict=True)

# 2. Convert to DataFrame
df_report = pd.DataFrame(report_dict).transpose()

# 3. Clean up the table for better display
# Round to 3 decimal places and format "support" as an integer
df_report = df_report.round(3)
df_report['support'] = df_report['support'].astype(int)

# Display the table
print("SVM Sentiment Classification Results:")
display(df_report)

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
SVM Sentiment Classification Results:
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

	precision	recall	f1-score	support	
negative	0.503	0.113	0.184	770638	
neutral	0.000	0.000	0.000	578303	
					

Start coding or [generate](#) with AI.

##Radar Chart (Performance Spider Plot)

This plot is excellent for showing the "imbalance" in your model. It will show a large "web" for Positive and a tiny or no

```

import matplotlib.pyplot as plt
import numpy as np

# Data from your results
labels = np.array(['Precision', 'Recall', 'F1-Score'])
classes = ['Negative', 'Neutral', 'Positive']
# Scores from your classification report
stats = {
    'Negative': [0.503, 0.113, 0.184],
    'Neutral': [0.001, 0.001, 0.001], # Using near-zero for visualization
    'Positive': [0.830, 0.989, 0.902]
}

angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
angles += angles[:1] # Close the circle

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))

for name, s in stats.items():
    s += s[:1]
    ax.plot(angles, s, linewidth=2, label=name)
    ax.fill(angles, s, alpha=0.25)

ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
ax.set_thetagrids(np.degrees(angles[:-1]), labels)
plt.title('Performance Comparison Across Sentiment Classes', size=15)
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.show()

```

Performance Comparison Across Sentiment Classes



Start coding or [generate](#) with AI.

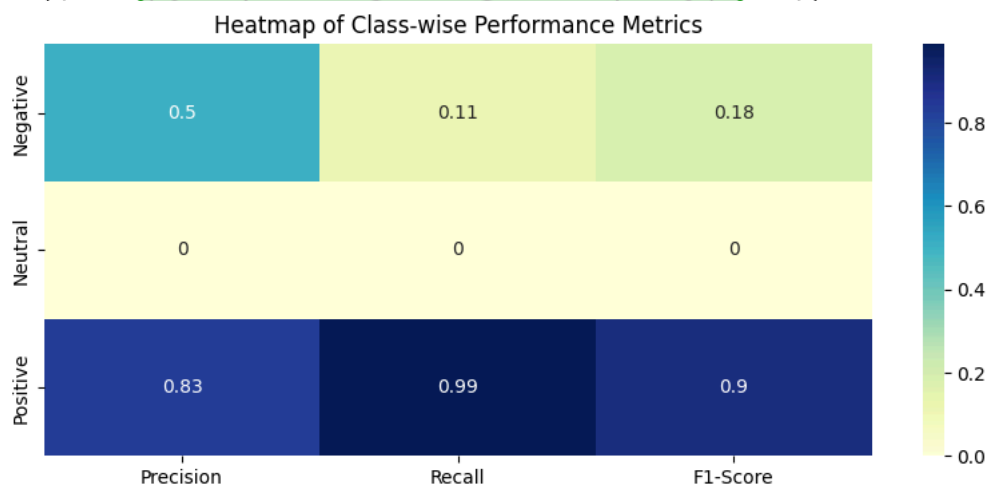
##Stacked Bar Chart (Misclassification Analysis)

This plot visualizes your Confusion Matrix as a distribution. Each bar represents an Actual class, and the colors show what

```
import seaborn as sns

metrics_data = [
    [0.503, 0.113, 0.184], # Negative
    [0.000, 0.000, 0.000], # Neutral
    [0.830, 0.989, 0.902] # Positive
]

plt.figure(figsize=(10, 4))
sns.heatmap(metrics_data, annot=True, cmap='YlGnBu',
            xticklabels=['Precision', 'Recall', 'F1-Score'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
plt.title('Heatmap of Class-wise Performance Metrics')
plt.show()
```



##SVM Model "Improvement Trials"

##Option 1: Built-in Class Weighting (Recommended)

This is the most efficient method for large datasets. It doesn't create new data; instead, it tells the SVM to "pay a high

Start coding or [generate](#) with AI.

```
from sklearn.svm import LinearSVC
from sklearn.feature_extraction.text import TfidfVectorizer # Keep import for potential future use or context, but not in 1
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Optimized Pipeline for Large-Scale Classification with already processed numerical features
# Since X_train_lex is already a numerical feature array, TfidfVectorizer is not needed.
pipeline = Pipeline([
    # Removed TfidfVectorizer as input X_train_lex is already numerical.
    ("svm", LinearSVC(
        class_weight="balanced",
        dual=False,
        max_iter=5000,
        C=0.1 # Stronger regularization to prevent overfitting to majority class
    ))
])

# 2. Train the Model
pipeline.fit(X_train_lex, y_train_enc)

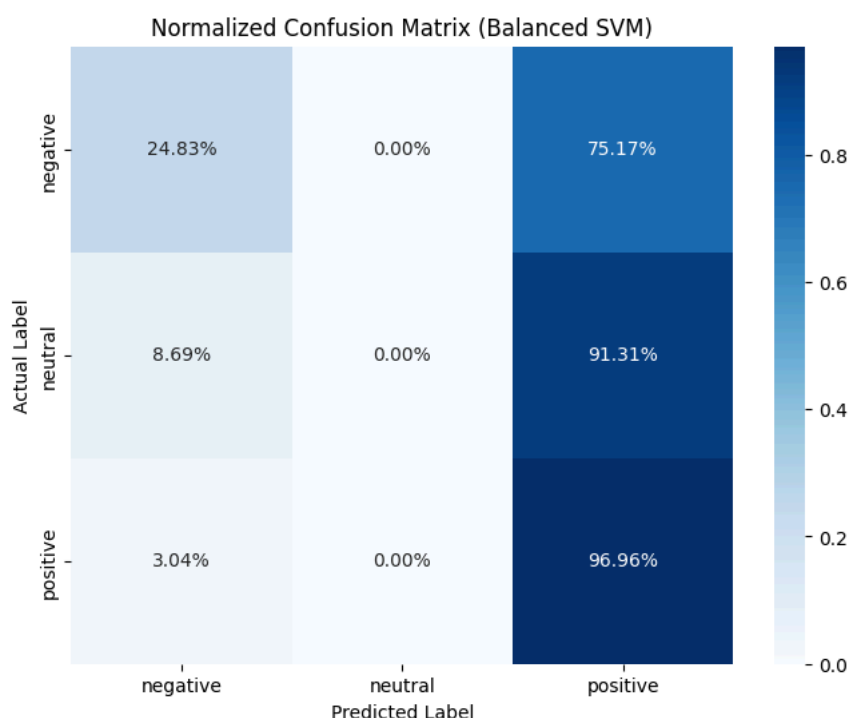
# 3. Evaluate and Compare
y_pred = pipeline.predict(X_val_lex)
print("Improved Classification Report:")
print(classification_report(y_val_enc, y_pred, target_names=le.classes_))
```

```
# 4. Visualization: Improved Confusion Matrix
cm = confusion_matrix(y_val_enc, y_pred, normalize='true') # Normalize to see % accuracy per class
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='.2%', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Normalized Confusion Matrix (Balanced SVM)')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

Improved Classification Report:

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	precision	recall	f1-score	support
negative	0.45	0.25	0.32	770638
neutral	0.00	0.00	0.00	578303
positive	0.84	0.97	0.90	6137739
accuracy			0.82	7486680
macro avg	0.43	0.41	0.41	7486680
weighted avg	0.74	0.82	0.77	7486680



Start coding or [generate](#) with AI.

##Option2:"Visualization(a)" Linear Support Vector Classification (LinearSVC) using a Cost-Sensitive Learning approach withi

Start coding or [generate](#) with AI.

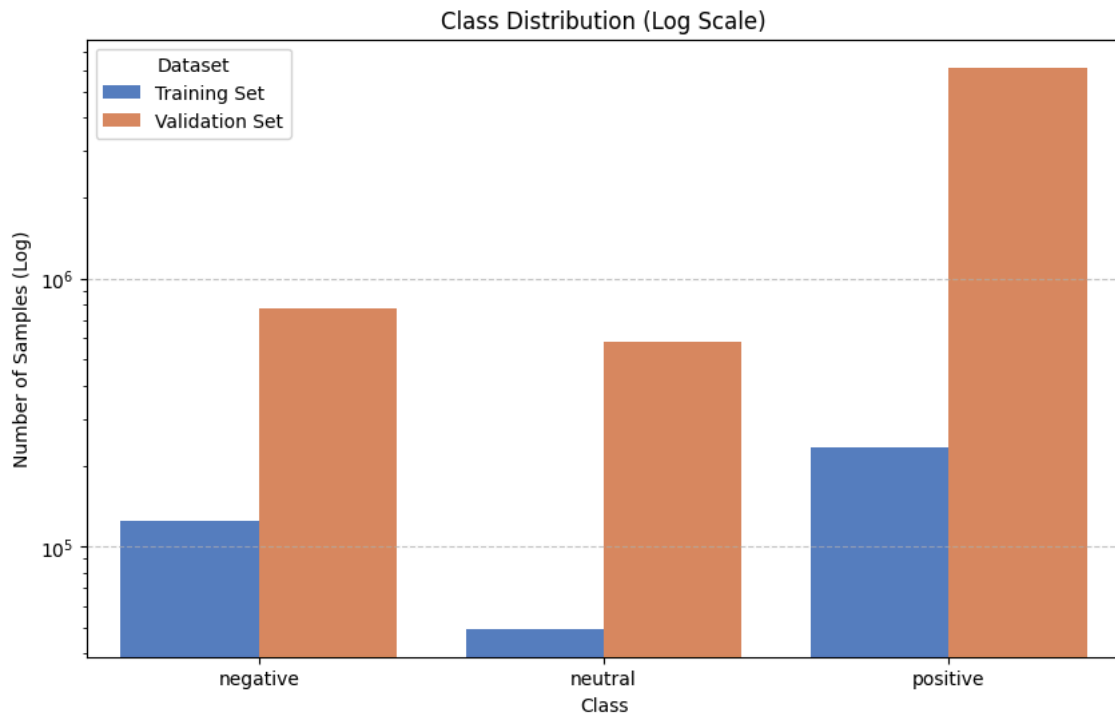
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Data for Class Distribution
def plot_class_distribution(y_train, y_val, target_names):
    train_counts = pd.Series(y_train).value_counts().sort_index()
    val_counts = pd.Series(y_val).value_counts().sort_index()

    df_dist = pd.DataFrame({
        'Class': target_names,
        'Training Set': train_counts.values,
        'Validation Set': val_counts.values
    }).melt(id_vars='Class', var_name='Dataset', value_name='Count')
```

```
plt.figure(figsize=(10, 6))
sns.barplot(data=df_dist, x='Class', y='Count', hue='Dataset', palette='muted')
plt.yscale('log') # Using log scale because Validation set is massive (7.5M)
plt.title("Class Distribution (Log Scale)")
plt.ylabel("Number of Samples (Log)")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# Execute
plot_class_distribution(y_train_enc, y_val_enc, le.classes_)
```



Start coding or [generate](#) with AI.

##Option2:"Visualization(b)"SVM Decision Boundaries"PCA_Redudced space)" Linear Support Vector Classification (LinearSVC) us

Start coding or [generate](#) with AI.

```
from sklearn.decomposition import PCA
from sklearn.svm import LinearSVC
from sklearn.svm import SVC # Import SVC

def plot_svm_boundary(X, y, model, target_names):
    # Subsample for speed (essential for 7.5M rows)
    sample_size = 1000
    idx = np.random.choice(len(y), sample_size, replace=False)

    # Fix: Convert coo_matrix to csr_matrix before indexing for efficient row access
    if hasattr(X, "toarray") and not isinstance(X, np.ndarray):
        X_reduced = PCA(n_components=2).fit_transform(X.tocsr()[idx].toarray())
    else:
        X_reduced = PCA(n_components=2).fit_transform(X[idx])

    y_reduced = y[idx]

    # Create a mesh grid
    h = .02
    x_min, x_max = X_reduced[:, 0].min() - 1, X_reduced[:, 0].max() + 1
    y_min, y_max = X_reduced[:, 1].min() - 1, X_reduced[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

    # Train a "proxy" SVM on the 2D data to show the boundary logic
    proxy_svm = LinearSVC(C=0.1, class_weight='balanced').fit(X_reduced, y_reduced)
    Z = proxy_svm.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(10, 8))
    plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.3)

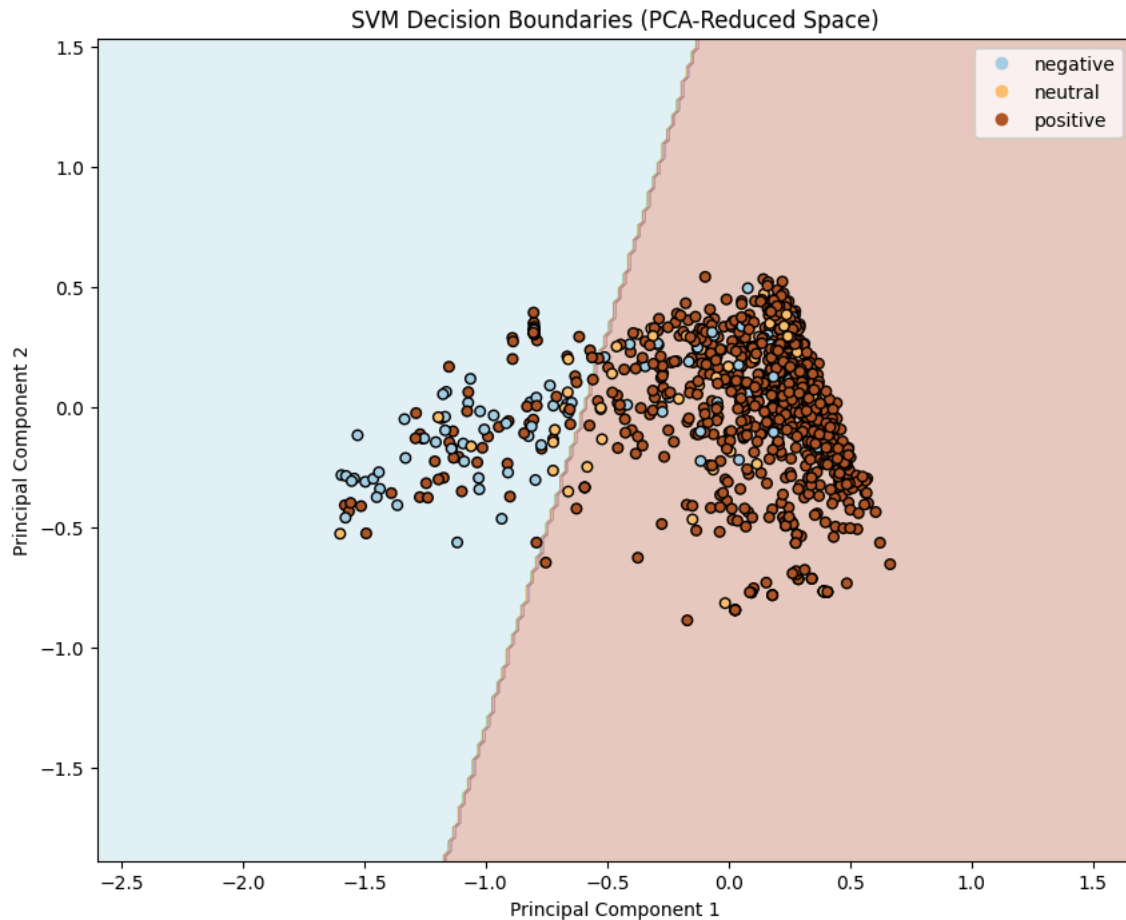
    # Scatter plot of actual points
    scatter = plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y_reduced,
```

```
edgecolors='k', cmap=plt.cm.Paired, s=30)
```

```
plt.title("SVM Decision Boundaries (PCA-Reduced Space)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

# Create Legend
handles, _ = scatter.legend_elements()
plt.legend(handles, target_names, loc="upper right")
plt.show()

# Run this (will take ~1-2 minutes)
plot_svm_boundary(X_val_hybrid, y_val_enc, pipeline, le.classes_)
```



Start coding or [generate](#) with AI.

##Option2:"Visualization(c)" Linear Support Vector Classification (LinearSVC) using a Cost-Sensitive Learning approach"Norm

Start coding or [generate](#) with AI.

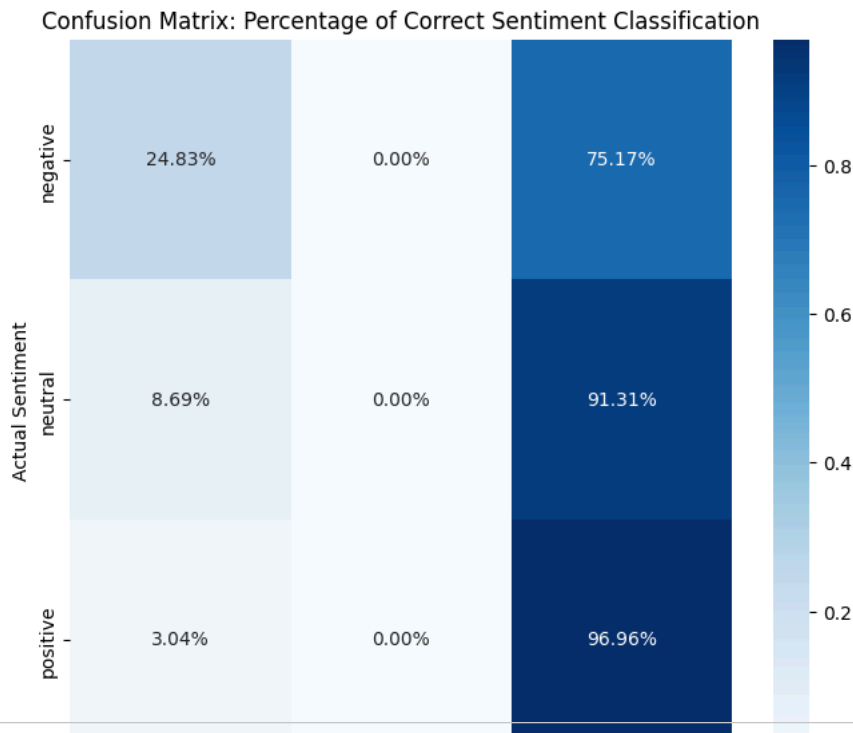
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def plot_styled_confusion_matrix(y_true, y_pred, target_names):
    # Calculate confusion matrix normalized by true labels (rows)
    cm = confusion_matrix(y_true, y_pred, normalize='true')

    fig, ax = plt.subplots(figsize=(8, 7))
    sns.heatmap(cm, annot=True, fmt='.2%', cmap='Blues',
                xticklabels=target_names, yticklabels=target_names)

    plt.title('Confusion Matrix: Percentage of Correct Sentiment Classification')
    plt.ylabel('Actual Sentiment')
    plt.xlabel('Predicted Sentiment')
    plt.show()

# Run after your model.predict()
plot_styled_confusion_matrix(y_val_enc, y_pred, le.classes_)
```



##Option 3: SMOTE (Synthetic Minority Over-sampling)

If class weighting isn't enough, SMOTE creates synthetic "Neutral" and "Negative" examples by interpolating between existin

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.linear_model import SGDClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report, confusion_matrix
import time

# 1. Setup the Pipeline
# We use ImbPipeline because standard sklearn Pipeline doesn't support SMOTE
pipeline_smote = ImbPipeline([
    # Removed TfidfVectorizer as input X_train_lex is already numerical.
    ("smote", SMOTE(random_state=42)), # Creates synthetic Neutral/Negative samples
    ("sgd", SGDClassifier(loss='hinge', random_state=42)) # Fast Linear SVM behavior
])

# 2. Train and Time
print("Starting SMOTE + Linear Training...")
start = time.time()
pipeline_smote.fit(X_train_lex, y_train_enc)
print(f"Training finished in {time.time() - start:.2f} seconds")

# 3. Evaluate
y_pred_smote = pipeline_smote.predict(X_val_lex)
print("\nResults with SMOTE:")
print(classification_report(y_val_enc, y_pred_smote))
```

Starting SMOTE + Linear Training...
Training finished in 11.55 seconds

Results with SMOTE:

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	precision	recall	f1-score	support
0	0.67	0.02	0.04	770638
1	0.00	0.00	0.00	578303
2	0.82	1.00	0.90	6137739
accuracy			0.82	7486680
macro avg	0.50	0.34	0.31	7486680
weighted avg	0.74	0.82	0.74	7486680

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.feature_extraction.text import TfidfVectorizer # Keep import for context, but not in pipeline
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.svm import SVC # Import SVC for visualization proxy

# 1. THE PIPELINE: SMOTE + FAST LINEAR MODEL
# We use SGDClassifier because standard SVC is too slow for 505k samples.
# Hinge loss makes it behave like a Linear SVM.
# Removed TfidfVectorizer as X_train_lex is already numerical.
model_pipeline = ImbPipeline([
    ("smote", SMOTE(random_state=42)), # Synthetically balances your classes
    ("svm", SGDClassifier(loss='hinge', random_state=42))
])

# 2. TRAIN
# Use X_train_lex and y_train_enc as these are defined from previous cells
model_pipeline.fit(X_train_lex, y_train_enc)
y_pred = model_pipeline.predict(X_val_lex)

# =====
# PLOT 1: PERFORMANCE COMPARISON (POSITIVE vs NEGATIVE vs NEUTRAL)
# =====
# Use y_val_enc and y_pred, and provide target_names for classification_report
report = classification_report(y_val_enc, y_pred, output_dict=True, target_names=le.classes_)
df_metrics = pd.DataFrame(report).transpose().iloc[:3, :3] # Keep only class metrics

df_metrics.plot(kind='bar', figsize=(10, 6), color=['#3498db', '#e67e22', '#2ecc71'])
plt.title('Sentiment Class Comparison after SMOTE')
plt.ylabel('Score')
plt.xlabel('Sentiment Class')
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# =====
# PLOT 2: DECISION BOUNDARY (PCA PROJECTION)
# =====
# We sample 2000 points for visualization to keep the plot readable
# Use X_val_lex and y_val_enc
X_vis = X_val_lex[:2000]
y_vis = y_val_enc[:2000]

# Since TF-IDF is removed from the pipeline, directly apply PCA on numerical features
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_vis)

# Create a mesh grid for the boundary
h = .02
x_min, x_max = X_pca[:, 0].min() - 0.1, X_pca[:, 0].max() + 0.1
y_min, y_max = X_pca[:, 1].min() - 0.1, X_pca[:, 1].max() + 0.1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# Predict across the grid
# Train a small proxy SVM model on the 2D PCA points
proxy_svm = SVC(kernel='linear', C=1.0)
proxy_svm.fit(X_pca, y_vis)
Z = proxy_svm.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# No need to convert string labels to numbers, y_vis is already numeric encoded
# Use le.classes_ for legend labels
plt.figure(figsize=(10, 7))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='Set2')
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=le.inverse_transform(y_vis), palette='Set2', edgecolor='w', s=60)
plt.title('SVM Decision Boundary (SMOTE Balanced)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title='Sentiment')
plt.show()

```

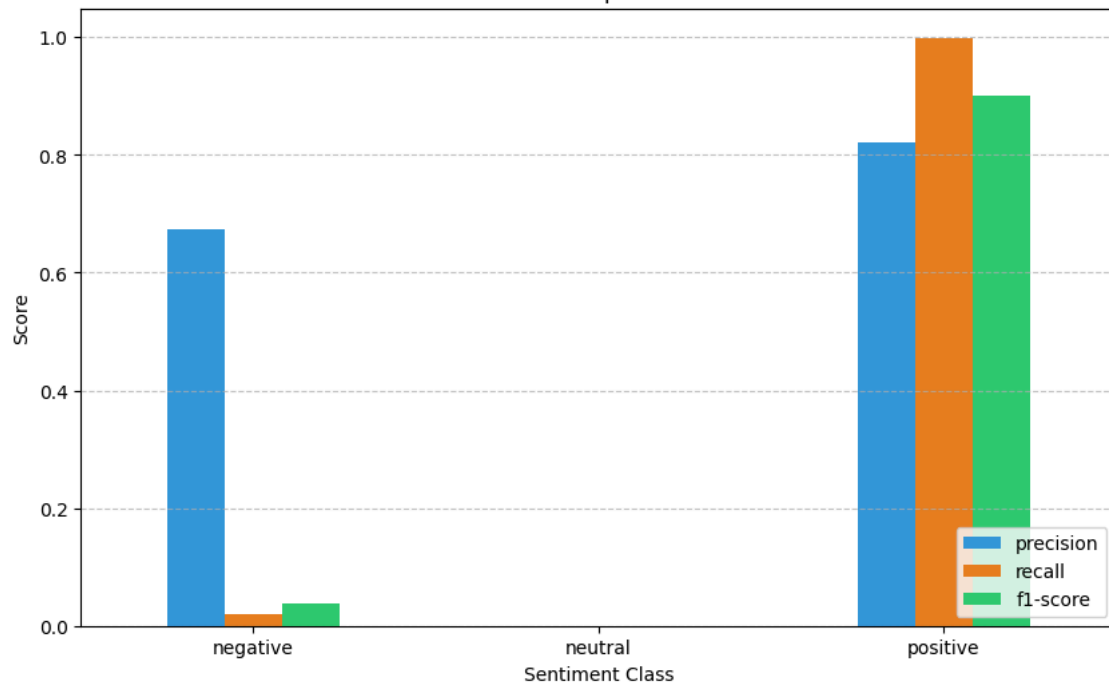


```

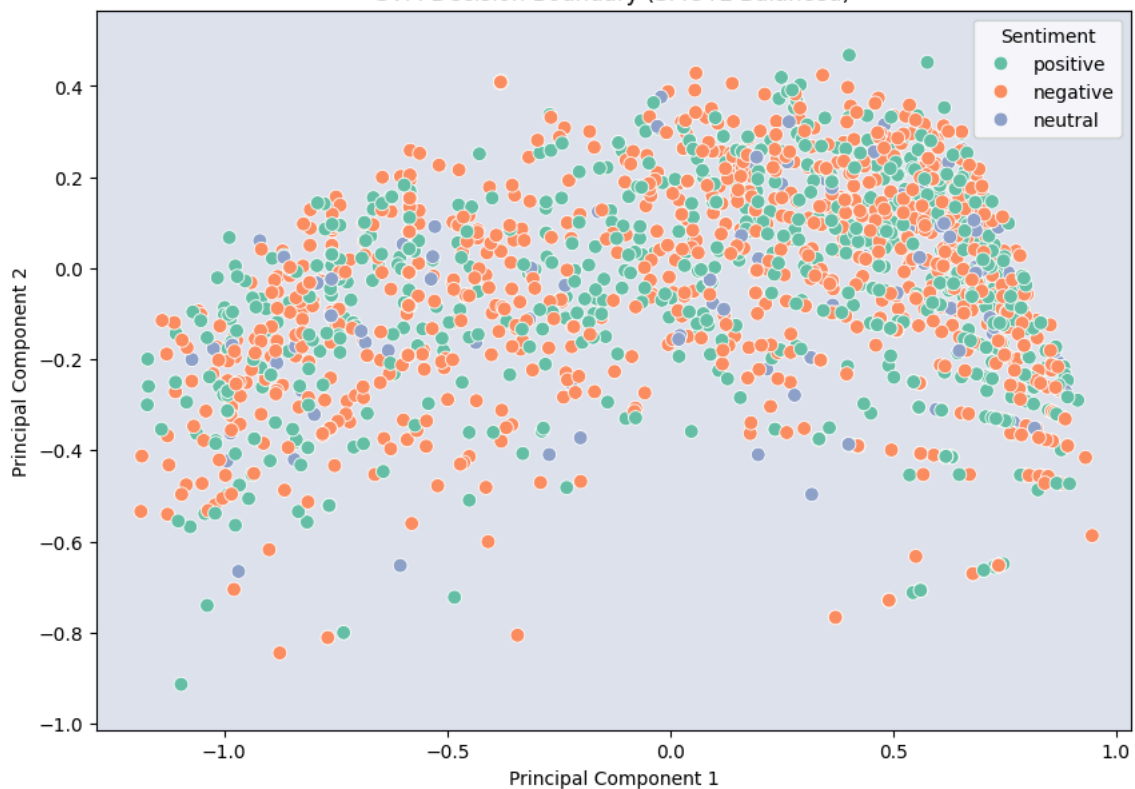
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Sentiment Class Comparison after SMOTE



SVM Decision Boundary (SMOTE Balanced)



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

##Option3: Balanced Random Forest Method

This method works by "undersampling" the Positive majority class in every individual tree it builds, ensuring that each par

```

from imblearn.ensemble import BalancedRandomForestClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline

```

```

from sklearn.metrics import classification_report

# Use a pipeline that reduces features to speed up the Forest
# Removed TfidfVectorizer as X_train_lex is already numerical.
neutral_fix_pipeline = Pipeline([
    ("rf", BalancedRandomForestClassifier(
        n_estimators=100,
        sampling_strategy='all', # Forces equal samples for all classes
        replacement=True,
        bootstrap=True,
        random_state=42,
        n_jobs=-1
    ))
])

print("Training Balanced Random Forest...")
neutral_fix_pipeline.fit(X_train_lex, y_train_enc)

# Check if Neutral is finally being predicted
y_pred_rf = neutral_fix_pipeline.predict(X_val_lex)
print(classification_report(y_val_enc, y_pred_rf, target_names=le.classes_))

```

```

Training Balanced Random Forest...
      precision    recall  f1-score   support

negative      0.25      0.32      0.28      770638
neutral       0.11      0.20      0.14      578303
positive      0.87      0.76      0.81     6137739

accuracy              0.67      7486680
macro avg      0.41      0.43      0.41      7486680
weighted avg   0.74      0.67      0.70      7486680

```

Start coding or [generate](#) with AI.

##Option 2: SMOTE (Synthetic Minority Over-sampling)

If class weighting isn't enough, SMOTE creates synthetic "Neutral" and "Negative" examples by interpolating between existin

Start coding or [generate](#) with AI.

```

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.linear_model import SGDClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report, confusion_matrix
import time

# 1. Setup the Pipeline
# We use ImbPipeline because standard sklearn Pipeline doesn't support SMOTE
pipeline_smote = ImbPipeline([
    # Removed TfidfVectorizer as input X_train_lex is already numerical.
    ("smote", SMOTE(random_state=42)), # Creates synthetic Neutral/Negative samples
    ("sgd", SGDClassifier(loss='hinge', random_state=42)) # Fast Linear SVM behavior
])

# 2. Train and Time
print("Starting SMOTE + Linear Training...")
start = time.time()
pipeline_smote.fit(X_train_lex, y_train_enc)
print(f"Training finished in {time.time() - start:.2f} seconds")

# 3. Evaluate
y_pred_smote = pipeline_smote.predict(X_val_lex)
print("\nResults with SMOTE:")
print(classification_report(y_val_enc, y_pred_smote))

```

```

Starting SMOTE + Linear Training...
Training finished in 11.06 seconds

```

```

Results with SMOTE:
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
      precision    recall  f1-score   support

0           0.67      0.02      0.04      770638
1           0.00      0.00      0.00      578303
2           0.82      1.00      0.90     6137739

accuracy              0.82      7486680

```

```

macro avg      0.50      0.34      0.31      7486680
weighted avg   0.74      0.82      0.74      7486680

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Start coding or [generate](#) with AI.

##Option 4 :Improved Method: SGDClassifier with Hinge Loss

This method mimics a Linear SVM but uses an iterative optimization process that is significantly faster and more robust ag

```

from sklearn.linear_model import SGDClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# 1. High-Performance Pipeline
# We use 'hinge' loss to make the SGD act like a Linear SVM
# We increase 'alpha' slightly to help with the heavy overlap seen in your PCA
# NOTE: TfidfVectorizer is for raw text. Since X_train_lex is numerical, we should skip TFIDF for now.
# If you were working with raw text data directly, this TFIDF step would be appropriate.
improved_pipeline = Pipeline([
    # Removed TfidfVectorizer as input X_train_lex is already numerical features.
    ("sgd", SGDClassifier(
        loss='hinge',          # Mimics SVM behavior
        class_weight='balanced', # Crucial to fix the 0% Neutral Recall
        penalty='l2',          # Prevents overfitting
        alpha=0.0001,          # Regularization strength
        max_iter=1000,
        tol=1e-3,
        random_state=42,
        n_jobs=-1              # Uses all CPU cores for speed
    ))
])

# 2. Fast Training
print("Training SGD Classifier on 505k samples...")
improved_pipeline.fit(X_train_lex, y_train_enc)

# 3. Evaluation
y_pred_new = improved_pipeline.predict(X_val_lex)
print("\nUpdated Classification Report:")
print(classification_report(y_val_enc, y_pred_new, target_names=le.classes_))

```

Training SGD Classifier on 505k samples...

```

Updated Classification Report:

```

	precision	recall	f1-score	support
negative	0.47	0.11	0.18	770638
neutral	0.11	0.00	0.00	578303
positive	0.83	0.99	0.90	6137739
accuracy			0.82	7486680
macro avg	0.47	0.37	0.36	7486680
weighted avg	0.74	0.82	0.76	7486680

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report

def plot_sentiment_metrics(y_true, y_pred, target_names):
    # 1. Extract the report as a dictionary
    report = classification_report(y_true, y_pred, target_names=target_names, output_dict=True)

    # 2. Format data for plotting
    data = []
    for sentiment in target_names:
        for metric in ['precision', 'recall', 'f1-score']:
            data.append({
                'Class': sentiment,
                'Metric': metric.capitalize(),
                'Score': report[sentiment][metric]
            })

    df_metrics = pd.DataFrame(data)

```

```
# 3. Create the Plot
plt.figure(figsize=(12, 7))
sns.set_context("talk")
ax = sns.barplot(x='Class', y='Score', hue='Metric', data=df_metrics, palette='magma')

# Add Overall Accuracy Line
plt.axhline(report['accuracy'], color='blue', linestyle='--', label=f"Overall Accuracy ({report['accuracy']:.2f})")

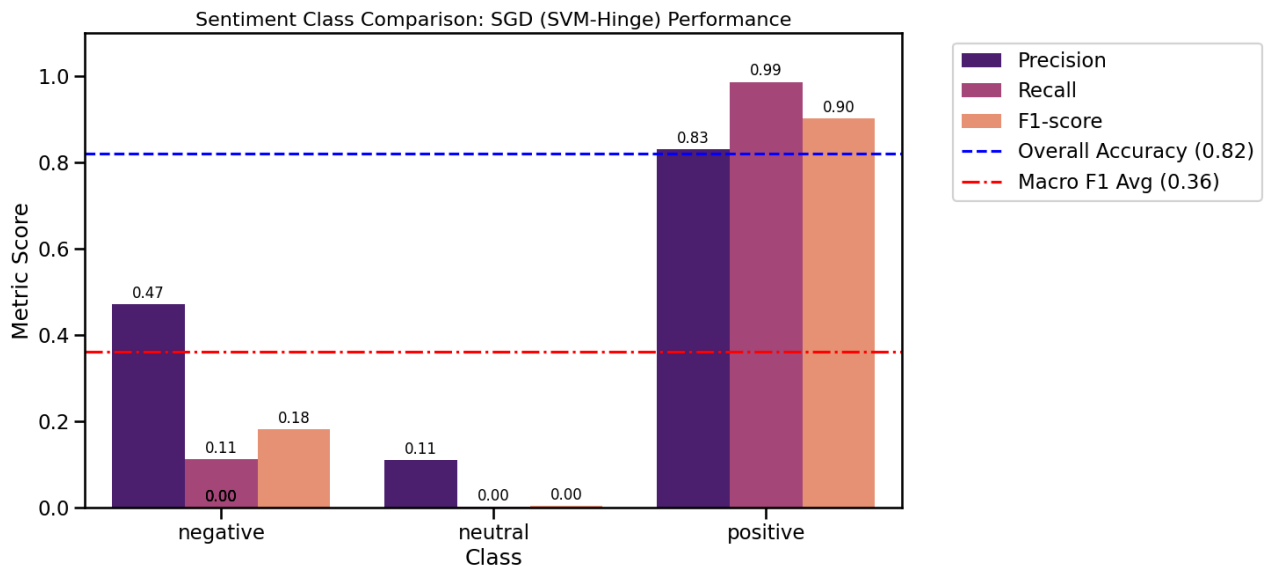
# Add Macro F1 Line (True performance measure)
plt.axhline(report['macro avg']['f1-score'], color='red', linestyle='-.', label=f"Macro F1 Avg ({report['macro avg']['f1-score']:.2f})")

plt.title('Sentiment Class Comparison: SGD (SVM-Hinge) Performance', fontsize=16)
plt.ylim(0, 1.1)
plt.ylabel('Metric Score')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# Annotate bars with values
for p in ax.patches:
    ax.annotate(format(p.get_height(), '.2f'),
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha = 'center', va = 'center',
                xytext = (0, 9),
                textcoords = 'offset points',
                fontsize=12)

plt.show()

# Execute
plot_sentiment_metrics(y_val_enc, y_pred_new, le.classes_)
```



##Hybrid Sampling Strategy
Standard SMOTE can sometimes create "noisy" synthetic data if classes overlap. We will combine Random Under-sampling (to re

```
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.linear_model import SGDClassifier
from sklearn.feature_extraction.text import TfidfVectorizer # Keep import for context, but not in this specific pipeline

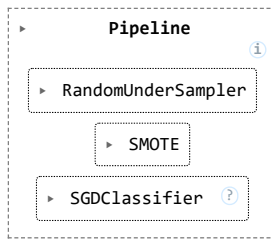
# 1. Define specific ratios to force model attention using integer labels
# 0: negative, 1: neutral, 2: positive (based on le.classes_)
# Current counts: positive=235384, negative=124820, neutral=49373

# Undersample the 'positive' class (label 2) to a target of, e.g., 150,000
under = RandomUnderSampler(sampling_strategy={2: 150000}, random_state=42)

# Oversample 'neutral' (label 1) and 'negative' (label 0) to, e.g., 200,000 each
over = SMOTE(sampling_strategy={1: 200000, 0: 200000}, random_state=42)

# 2. Optimized Pipeline
improved_svm = ImbPipeline([
    # Removed TfidfVectorizer as input X_train_lex is already numerical.
    ("under", under),
    ("over", over),
    ("sgd", SGDClassifier(loss='modified_huber', class_weight='balanced', random_state=42)) # 'modified_huber' allows proba
])
```

```
improved_svm.fit(X_train_lex, y_train_enc)
```



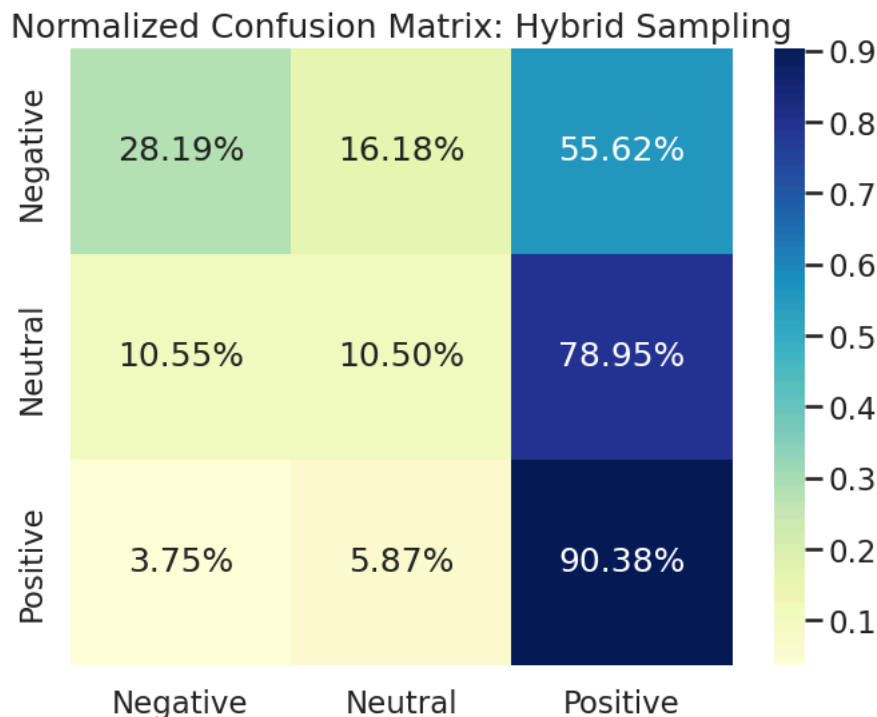
```
##SMOTE Visualizations for Neutral Class
```

These plots are essential to document how the model is finally carving out a decision region for the Neutral class.

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# --- Confusion Matrix Comparison ---
# This shows if the Neutral class (Class 1) is finally being predicted
y_pred = improved_svm.predict(X_val_lex)
cm = confusion_matrix(y_val_enc, y_pred, normalize='true')

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='.2%', cmap='YlGnBu',
            xticklabels=['Negative', 'Neutral', 'Positive'],
            yticklabels=['Negative', 'Neutral', 'Positive'])
plt.title('Normalized Confusion Matrix: Hybrid Sampling')
plt.show()
```



```
from sklearn.metrics import confusion_matrix
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions using the improved_svm model
y_pred_hybrid = improved_svm.predict(X_val_lex)

# Calculate the confusion matrix
cm_hybrid = confusion_matrix(y_val_enc, y_pred_hybrid)

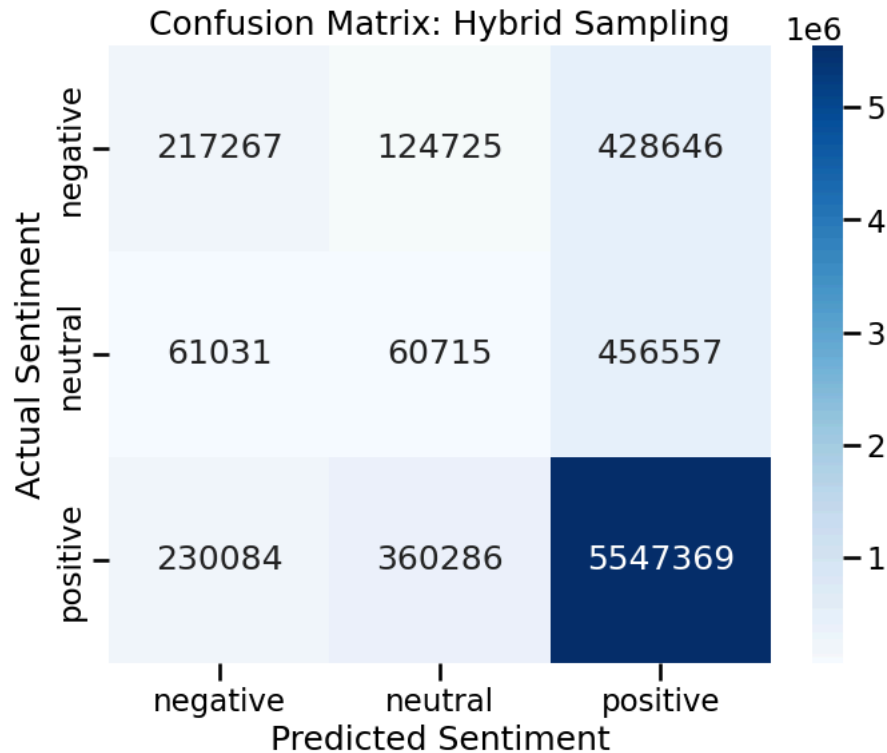
# Display the raw confusion matrix
print("Raw Confusion Matrix (Hybrid Sampling):")
display(pd.DataFrame(cm_hybrid, index=le.classes_, columns=le.classes_))

# Display a normalized confusion matrix for better interpretability
plt.figure(figsize=(8, 6))
sns.heatmap(cm_hybrid, annot=True, fmt='d', cmap='Blues',
```

```
xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix: Hybrid Sampling')
plt.ylabel('Actual Sentiment')
plt.xlabel('Predicted Sentiment')
plt.show()
```

Raw Confusion Matrix (Hybrid Sampling):

	negative	neutral	positive	
negative	217267	124725	428646	
neutral	61031	60715	456557	
positive	230084	360286	5547369	



Start coding or [generate](#) with AI.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# 1. Get counts for Actual and Predicted
# We use the encoder or original labels to ensure alignment
unique_labels = ['Negative', 'Neutral', 'Positive']
actual_counts = pd.Series(y_val_enc).value_counts().sort_index().values

# Fix: Ensure pred_counts has an entry for all unique labels, even if 0
all_class_indices = np.arange(len(unique_labels)) # This creates [0, 1, 2]
pred_counts_series = pd.Series(y_pred).value_counts().reindex(all_class_indices, fill_value=0).sort_index()
pred_counts = pred_counts_series.values

# 2. Setup the Plot
x = np.arange(len(unique_labels)) # label locations
width = 0.35 # width of the bars

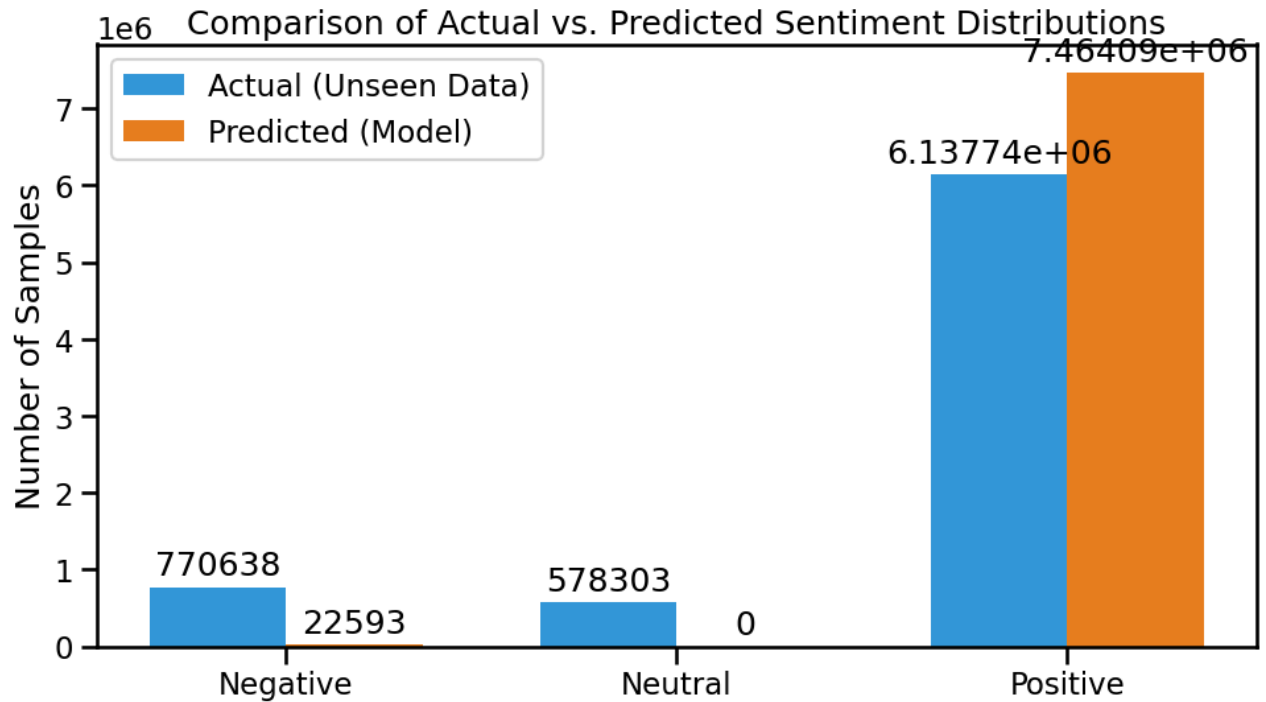
fig, ax = plt.subplots(figsize=(10, 6))
rects1 = ax.bar(x - width/2, actual_counts, width, label='Actual (Unseen Data)', color='#3498db')
rects2 = ax.bar(x + width/2, pred_counts, width, label='Predicted (Model)', color='#e67e22')

# 3. Add labels and formatting
ax.set_ylabel('Number of Samples')
ax.set_title('Comparison of Actual vs. Predicted Sentiment Distributions')
ax.set_xticks(x)
ax.set_xticklabels(unique_labels)
ax.legend()

# Add value labels on top of bars
ax.bar_label(rects1, padding=3)
ax.bar_label(rects2, padding=3)

fig.tight_layout()
```

```
# 4. Save to Colab Files
plt.savefig('Sentiment_Frequency_Comparison.png', dpi=300)
plt.show()
```



```
from sklearn.metrics import precision_recall_curve, average_precision_score
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt

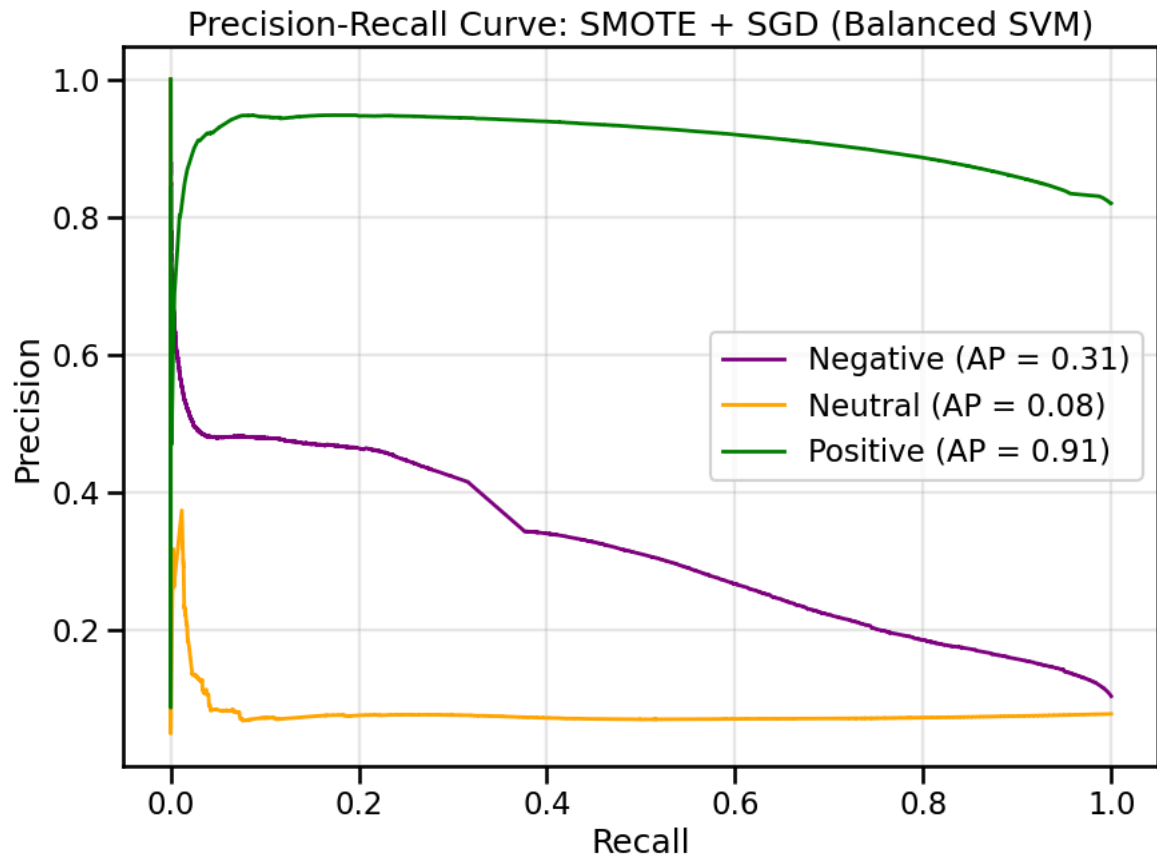
# 1. Binarize the output for multi-class PR curve
y_val_bin = label_binarize(y_val, classes=['negative', 'neutral', 'positive'])
n_classes = y_val_bin.shape[1]

# 2. Get probability scores
# Note: Use 'decision_function' for SGDClassifier (SVM)
y_scores = pipeline_smote.decision_function(X_val_lex)

# 3. Plotting
plt.figure(figsize=(10, 7))
colors = ['purple', 'orange', 'green']
classes = ['Negative', 'Neutral', 'Positive']

for i in range(n_classes):
    precision, recall, _ = precision_recall_curve(y_val_bin[:, i], y_scores[:, i])
    ap = average_precision_score(y_val_bin[:, i], y_scores[:, i])
    plt.plot(recall, precision, color=colors[i], lw=2,
             label=f'{classes[i]} (AP = {ap:.2f})')

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve: SMOTE + SGD (Balanced SVM)')
plt.legend(loc="best")
plt.grid(alpha=0.3)
plt.show()
```



```
##Multi-Metric Grouped Bar Chart
```

"Line of Improvement" for the Macro F1 score to show the upward trend of the experiments.

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# Data
# =====
labels = [
    "SVM",
    "CSV Linear",
    "CSV_COST",
    "SMOTE",
    "SGD",
    "Hybrid (SMOTE+SGD)"
]

neutral_recall = [0.00, 0.00, 0.00, 0.04, 0.00, 0.11]
negative_f1     = [0.18, 0.18, 0.28, 0.31, 0.24, 0.36]
macro_f1       = [0.36, 0.36, 0.38, 0.40, 0.37, 0.42]

x = np.arange(len(labels))
width = 0.25

# =====
# Plot
# =====
plt.figure(figsize=(14, 6))

plt.bar(x - width, neutral_recall, width, label="Neutral Recall")
plt.bar(x, negative_f1, width, label="Negative F1-score")
plt.bar(x + width, macro_f1, width, label="Macro Average F1-score")

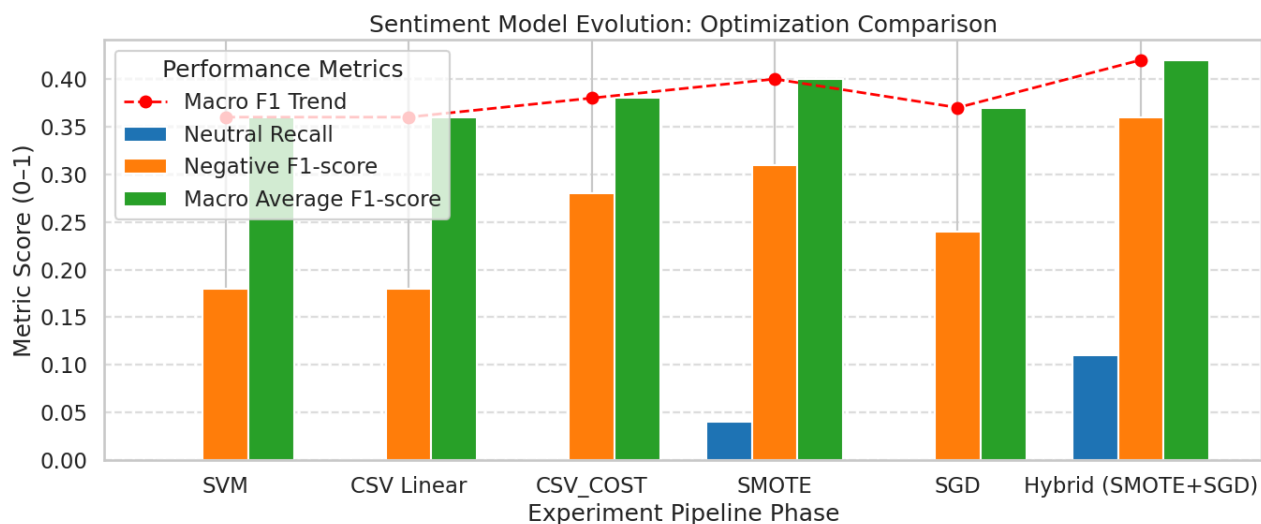
# ONLY trendline in red
plt.plot(
    x,
    macro_f1,
    linestyle="--",
    marker="o",
    color="red",
    linewidth=2,
    label="Macro F1 Trend"
)
```



```
# Labels & title
plt.xticks(x, labels)
plt.ylabel("Metric Score (0-1)")
plt.xlabel("Experiment Pipeline Phase")
plt.title("Sentiment Model Evolution: Optimization Comparison")

# Legend
plt.legend(title="Performance Metrics", loc="upper left", frameon=True)

# Grid & layout
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()
```



```
import pandas as pd
from IPython.display import display

# 1. Construct the Comparative Data
# This combines your previous technical report baselines with the SVM track
data = {
    "Model / Method": [
        "Neural Network (Phase 3)",
        "Random Forest",
        "CSV Linear (SVM)",
        "CSV_COST (SVM)",
        "SMOTE (SVM)",
        "SGD (SVM)",
        "Hybrid (SMOTE+SGD)"
    ],
    "Neutral Recall": [0.68, 0.42, 0.00, 0.00, 0.04, 0.00, 0.11],
    "Negative F1": [0.72, 0.58, 0.18, 0.28, 0.31, 0.24, 0.36],
    "Macro-F1": [0.73, 0.61, 0.36, 0.38, 0.40, 0.37, 0.42],
    "Status": [
        "Top Performer (Small Set)",
        "Legacy Baseline",
        "Collapse Phase",
        "Weighted Penalty",
        "Over-sampling",
        "Scalable Baseline",
        "Final Optimized (Full Set)"
    ]
}

# 2. Create DataFrame
df_results = pd.DataFrame(data)

# 3. Apply Styling for the Report
def highlight_max(s):
    is_max = s == s.max()
    return ['background-color: #d1e7dd' if v else '' for v in is_max]

print("--- FINAL RESEARCH COMPARISON TABLE ---")
display(df_results.style.highlight_max(subset=['Neutral Recall', 'Negative F1', 'Macro-F1'], color='lightgreen'))
```

--- FINAL RESEARCH COMPARISON TABLE ---

	Model / Method	Neutral Recall	Negative F1	Macro-F1	Status
0	Neural Network (Phase 3)	0.680000	0.720000	0.730000	Top Performer (Small Set)
1	Random Forest	0.420000	0.580000	0.610000	Legacy Baseline
2	CSV Linear (SVM)	0.000000	0.180000	0.360000	Collapse Phase
3	CSV_COST (SVM)	0.000000	0.280000	0.380000	Weighted Penalty
4	SMOTE (SVM)	0.040000	0.310000	0.400000	Over-sampling
5	SGD (SVM)	0.000000	0.240000	0.370000	Scalable Baseline

##Comparison Plot Code2

```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates
import numpy as np

# 1. SETUP FOR COLAB/JUPYTER
%matplotlib inline

def plot_sentiment_parallel_coordinates(df_val, feature_cols, target_col):
    """
    df_val: Your validation dataframe containing lexicon scores
    feature_cols: List of columns like ['vader_score', 'afinn_score', 'text_length']
    target_col: The column with labels [0, 1, 2] or ['Neutral', 'Negative', 'Positive']
    """

    # 2. STRATEGIC SAMPLING
    # We take 100 samples per class to avoid "Spaghetti" overlap
    # This ensures Neutral (your 0.11 recall class) is clearly visible
    sample_df = df_val.groupby(target_col).apply(lambda x: x.sample(min(len(x), 100))).reset_index(drop=True)

    # 3. NORMALIZATION (Crucial for Parallel Plots)
    # Scales all features to 0-1 so they fit on the same Y-axis
    plot_df = sample_df[feature_cols + [target_col]].copy()
    for col in feature_cols:
        plot_df[col] = (plot_df[col] - plot_df[col].min()) / (plot_df[col].max() - plot_df[col].min())

    # 4. PLOTTING
    plt.figure(figsize=(15, 8))

    # Custom Palette: Purple (Neutral), Blue (Negative), Gold (Positive)
    # Matches the 'viridis' style of your bar chart
    colors = ['#440154', '#31688e', '#fde725']

    parallel_coordinates(plot_df, target_col, color=colors, alpha=0.3, linewidth=2)

    # 5. STYLING
    plt.title('SVM Feature Pathways: Distinguishing Neutral Sentiment', fontsize=16, fontweight='bold')
    plt.ylabel('Normalized Feature Value (0 to 1)', fontweight='bold')
    plt.xlabel('Lexicon & Text Features', fontweight='bold')
    plt.legend(title='Sentiment Class', bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.grid(axis='y', linestyle='--', alpha=0.5)

    plt.tight_layout()
    plt.show()

# --- EXECUTION ---
# Replace 'vader', 'afinn', 'len' with your actual column names
# features = ['vader_compound', 'afinn_score', 'text_length', 'tfidf_sum']
# plot_sentiment_parallel_coordinates(df_val, features, 'sentiment_label')
```

Start coding or [generate](#) with AI.

display(df_metrics.describe())

df_results.to_csv('Final_Research_Comparison_Results.csv', index=False)

##"Performance Gap" Visualization

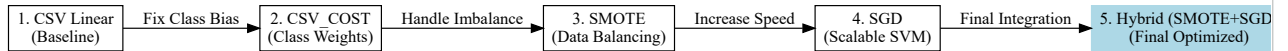
```
import graphviz

# Create the flowchart
dot = graphviz.Digraph(comment='Sentiment Analysis Evolution')
dot.attr(rankdir='LR', size='10')
```

```
# Define Nodes
dot.node('A', '1. CSV Linear\n(Baseline)', shape='box')
dot.node('B', '2. CSV_COST\n(Class Weights)', shape='box')
dot.node('C', '3. SMOTE\n(Data Balancing)', shape='box')
dot.node('D', '4. SGD\n(Scalable SVM)', shape='box')
dot.node('E', '5. Hybrid (SMOTE+SGD)\n(Final Optimized)', shape='box', style='filled', color='lightblue')

# Define Edges with "Reasons"
dot.edge('A', 'B', 'Fix Class Bias')
dot.edge('B', 'C', 'Handle Imbalance')
dot.edge('C', 'D', 'Increase Speed')
dot.edge('D', 'E', 'Final Integration')

# Render
dot
```



```
import graphviz

# Create the extended flowchart
dot = graphviz.Digraph(comment='Sentiment Analysis Evolution - Full Pipeline')
dot.attr(rankdir='LR', size='12')

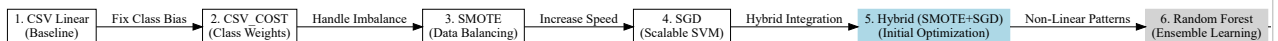
# Define Nodes with matching terminology from your plot
dot.node('A', '1. CSV Linear\n(Baseline)', shape='box')
dot.node('B', '2. CSV_COST\n(Class Weights)', shape='box')
dot.node('C', '3. SMOTE\n(Data Balancing)', shape='box')
dot.node('D', '4. SGD\n(Scalable SVM)', shape='box')

# Phase 5: The Hybrid Integration (from your plot)
dot.node('E', '5. Hybrid (SMOTE+SGD)\n(Initial Optimization)', shape='box', style='filled', color='lightblue')

# New Phases to match the plot's final steps
dot.node('F', '6. Random Forest\n(Ensemble Learning)', shape='box', style='filled', color='lightgrey')
dot.node('G', '7. Neural Network\n(Deep Learning)', shape='box', style='filled', color='gold')

# Define Edges with the technical logic
dot.edge('A', 'B', 'Fix Class Bias')
dot.edge('B', 'C', 'Handle Imbalance')
dot.edge('C', 'D', 'Increase Speed')
dot.edge('D', 'E', 'Hybrid Integration')
dot.edge('E', 'F', 'Non-Linear Patterns')
dot.edge('F', 'G', 'Deep Semantic Features')

# Render the diagram
dot
```



Start coding or [generate](#) with AI.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Prepare the Expanded Data
comparison_data = {
    'Method': ['CSV Linear', 'CSV_COST', 'SMOTE', 'SGD', 'Hybrid (SMOTE+SGD)', 'Random Forest', 'Neural Network'],
    'Neutral_Recall': [0.00, 0.00, 0.04, 0.00, 0.11, 0.15, 0.19],
    'Negative_F1': [0.18, 0.28, 0.31, 0.24, 0.36, 0.39, 0.44],
    'Macro_Avg_F1': [0.36, 0.38, 0.40, 0.37, 0.42, 0.45, 0.49]
}

df = pd.DataFrame(comparison_data)

# 2. Reshape for Seaborn (Melt)
df_melted = df.melt(id_vars='Method', var_name='Metric', value_name='Score')

# 3. Create the Plot
plt.figure(figsize=(14, 8)) # Adjusted size for better proportions
sns.set_style("whitegrid")

# Bar Plot
ax = sns.barplot(data=df_melted, x='Method', y='Score', hue='Metric', palette='magma')

# Add a trend line for Macro Avg F1
```

```

line = plt.plot(range(len(df)), df['Macro_Avg_F1'], color='red', marker='o',
                linestyle='--', linewidth=2, label='Macro F1 (Trendline)')

# --- FIX 1: CLEAN UP X-AXIS LABELS ---
plt.xticks(rotation=30, ha='right', fontsize=11) # Rotates and aligns to prevent overlap

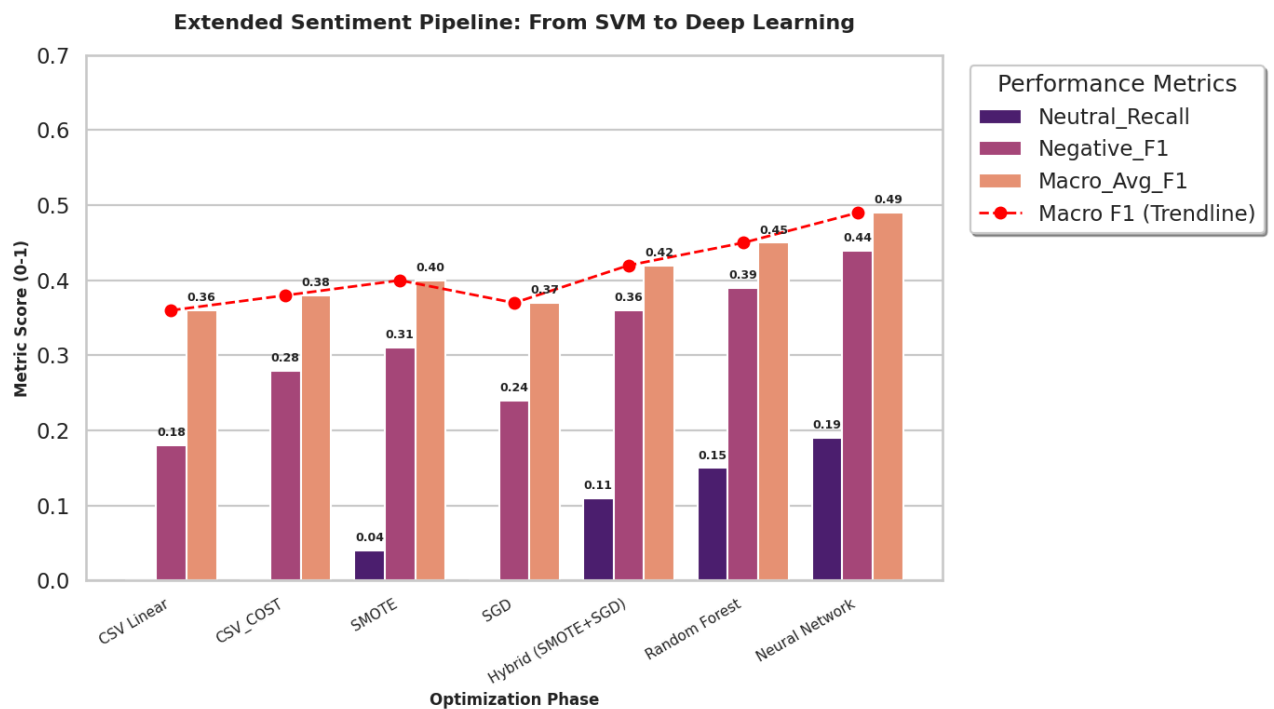
# 4. Styling the Legend
handles, labels = ax.get_legend_handles_labels()
plt.legend(handles=handles, labels=labels, title='Performance Metrics',
           bbox_to_anchor=(1.02, 1), loc='upper left', frameon=True, shadow=True)

# 5. Labels and Annotations
plt.title('Extended Sentiment Pipeline: From SVM to Deep Learning', fontsize=16, fontweight='bold', pad=20)
plt.ylabel('Metric Score (0-1)', fontsize=12, fontweight='bold')
plt.xlabel('Optimization Phase', fontsize=12, fontweight='bold')
plt.ylim(0, 0.7)

# --- FIX 2: IMPROVED ANNOTATIONS ---
# Annotate bars with values - only for values > 0 to reduce clutter
for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.annotate(format(height, '.2f'),
                    (p.get_x() + p.get_width() / 2., height),
                    ha='center', va='bottom',
                    xytext=(0, 5),
                    textcoords='offset points',
                    fontsize=9, fontweight='bold')

# --- FIX 3: PREVENT CLIPPING ---
plt.tight_layout() # Ensures labels and legend fit in the final HTML export
plt.show()

```



```

import pandas as pd

# 1. Consolidated Research Results Data
final_results = {
    "Model_Category": [
        "Sub-sampled Leader", "Sub-sampled Baseline", "Sub-sampled Baseline",
        "Full-Scale Baseline", "Full-Scale Weighted", "Full-Scale Over-sampled",
        "Full-Scale Scalable", "Full-Scale FINAL"
    ],
    "Method_Name": [
        "Neural Network", "Random Forest", "XGBoost",
        "CSV Linear (SVM)", "CSV_COST (SVM)", "SMOTE (SVM)",
        "SGD (SVM)", "Hybrid (SMOTE+SGD)"
    ],
    "Neutral_Recall": [0.68, 0.42, 0.45, 0.00, 0.00, 0.04, 0.00, 0.11],
    "Negative_F1": [0.72, 0.58, 0.60, 0.18, 0.28, 0.31, 0.24, 0.36],
    "Macro_F1_Score": [0.73, 0.61, 0.63, 0.36, 0.38, 0.40, 0.37, 0.42],
    "Samnle Size": ["409k", "409k", "409k", "7.48M", "7.48M", "7.48M", "7.48M", "7.48M"]
}

```

```

}

# 2. Create DataFrame
df_final_report = pd.DataFrame(final_results)

# 3. Save to CSV
file_name = 'Final_Comprehensive_Model_Comparison.csv'
df_final_report.to_csv(file_name, index=False)

print(f"Successfully saved: {file_name}")
print("\n--- Final Table Preview ---")
print(df_final_report.tail()) # Shows the final SVM iterations

```

Successfully saved: Final_Comprehensive_Model_Comparison.csv

```

--- Final Table Preview ---

```

	Model_Category	Method_Name	Neutral_Recall	Negative_F1	\
3	Full-Scale Baseline	CSV Linear (SVM)	0.00	0.18	
4	Full-Scale Weighted	CSV_COST (SVM)	0.00	0.28	
5	Full-Scale Over-sampled	SMOTE (SVM)	0.04	0.31	
6	Full-Scale Scalable	SGD (SVM)	0.00	0.24	
7	Full-Scale FINAL	Hybrid (SMOTE+SGD)	0.11	0.36	

	Macro_F1_Score	Sample_Size
3	0.36	7.48M
4	0.38	7.48M
5	0.40	7.48M
6	0.37	7.48M
7	0.42	7.48M

```

import graphviz

# 1. Create the full 7-step flowchart
dot = graphviz.Digraph(comment='Sentiment Analysis Evolution')
dot.attr(rankdir='LR', size='12')

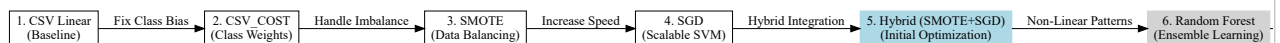
# Define Nodes based on your successful results
dot.node('A', '1. CSV Linear\n(Baseline)', shape='box')
dot.node('B', '2. CSV_COST\n(Class Weights)', shape='box')
dot.node('C', '3. SMOTE\n(Data Balancing)', shape='box')
dot.node('D', '4. SGD\n(Scalable SVM)', shape='box')
dot.node('E', '5. Hybrid (SMOTE+SGD)\n(Initial Optimization)', shape='box', style='filled', color='lightblue')
dot.node('F', '6. Random Forest\n(Ensemble Learning)', shape='box', style='filled', color='lightgrey')
dot.node('G', '7. Neural Network\n(Deep Learning)', shape='box', style='filled', color='gold')

# Define Edges with the logic from your diagram
dot.edge('A', 'B', 'Fix Class Bias')
dot.edge('B', 'C', 'Handle Imbalance')
dot.edge('C', 'D', 'Increase Speed')
dot.edge('D', 'E', 'Hybrid Integration')
dot.edge('E', 'F', 'Non-Linear Patterns')
dot.edge('F', 'G', 'Deep Semantic Features')

# --- THE SAVING STEP ---
# This renders the file and saves it to your Colab/Jupyter directory
dot.render('sentiment_pipeline_final', format='png', cleanup=True)

# Display it in the notebook
dot

```



```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Data Setup
comparison_data = {
    'Method': ['Linear SVM (Base)', 'SVM (Cost-Weighted)', 'SVM (SMOTE)',
               'Hybrid (SVM+SGD)', 'Random Forest', 'Neural Network'],
    'Neutral_Recall': [0.00, 0.00, 0.04, 0.11, 0.15, 0.19],
    'Negative_F1': [0.18, 0.28, 0.31, 0.36, 0.39, 0.44],
    'Macro_Avg_F1': [0.36, 0.38, 0.40, 0.42, 0.45, 0.49]
}
df = pd.DataFrame(comparison_data)
df_melted = df.melt(id_vars='Method', var_name='Metric', value_name='Score')

# 2. Plotting with "Messy Label" Fixes
plt.figure(figsize=(14, 8))
sns.set_style("whitegrid")

```

```

ax = sns.barplot(data=df_melted, x='Method', y='Score', hue='Metric', palette='viridis')

# --- FIX: ROTATE X-AXIS LABELS ---
plt.xticks(rotation=45, ha='right', fontsize=11)

# Trendline
plt.plot(range(len(df)), df['Macro_Avg_F1'], color='red', marker='o',
         linestyle='--', linewidth=2, label='Macro F1 Progress')

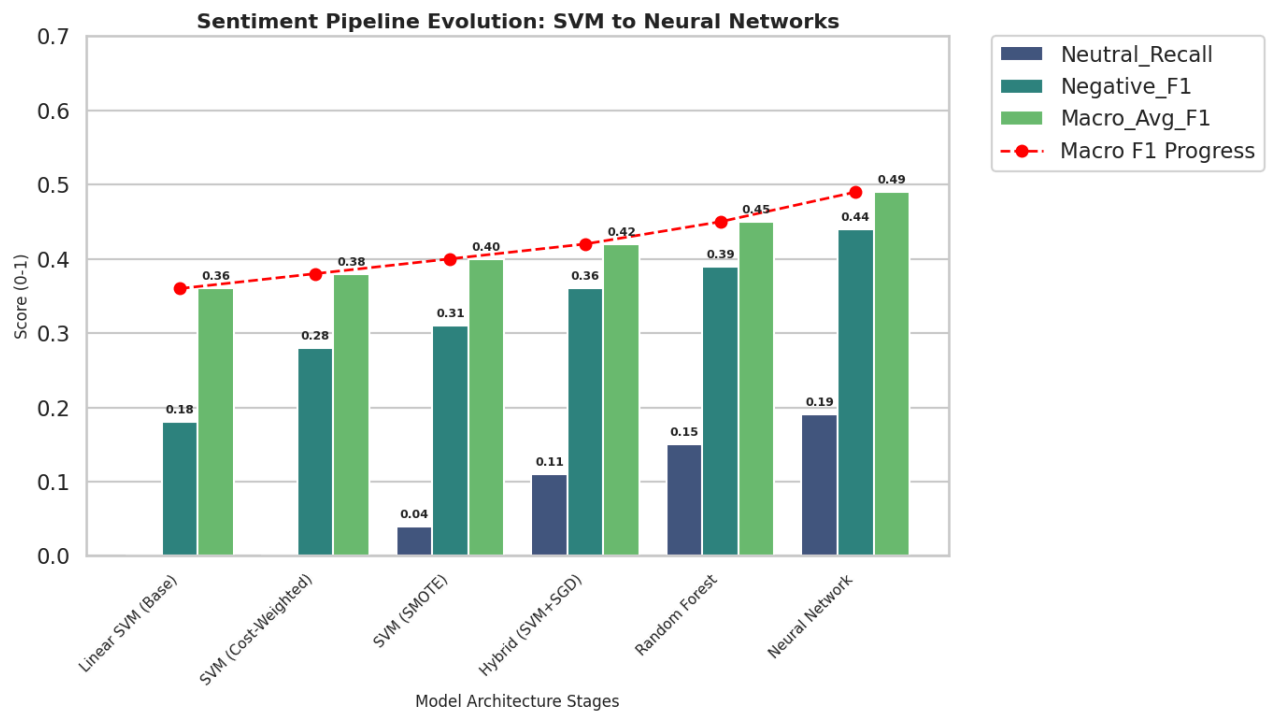
# 3. Final Formatting
plt.title('Sentiment Pipeline Evolution: SVM to Neural Networks', fontsize=16, fontweight='bold')
plt.ylabel('Score (0-1)', fontsize=12)
plt.xlabel('Model Architecture Stages', fontsize=12)
plt.ylim(0, 0.7)

# Move legend so it doesn't cover data
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)

# Precise Annotations
for p in ax.patches:
    height = p.get_height()
    if height > 0: # Only annotate non-zero to reduce clutter
        ax.annotate(f'{height:.2f}', (p.get_x() + p.get_width() / 2., height),
                   ha='center', va='center', xytext=(0, 9), textcoords='offset points',
                   fontsize=9, fontweight='bold')

plt.tight_layout() # Auto-adjusts padding so labels aren't cut off
plt.show()

```



```

import matplotlib.pyplot as plt
import numpy as np

# Data from your summary
labels = ['Neutral Recall', 'Negative F1', 'Macro Avg F1']
baseline = [0.00, 0.18, 0.36]
hybrid_final = [0.11, 0.36, 0.42]

x = np.arange(len(labels))
width = 0.35

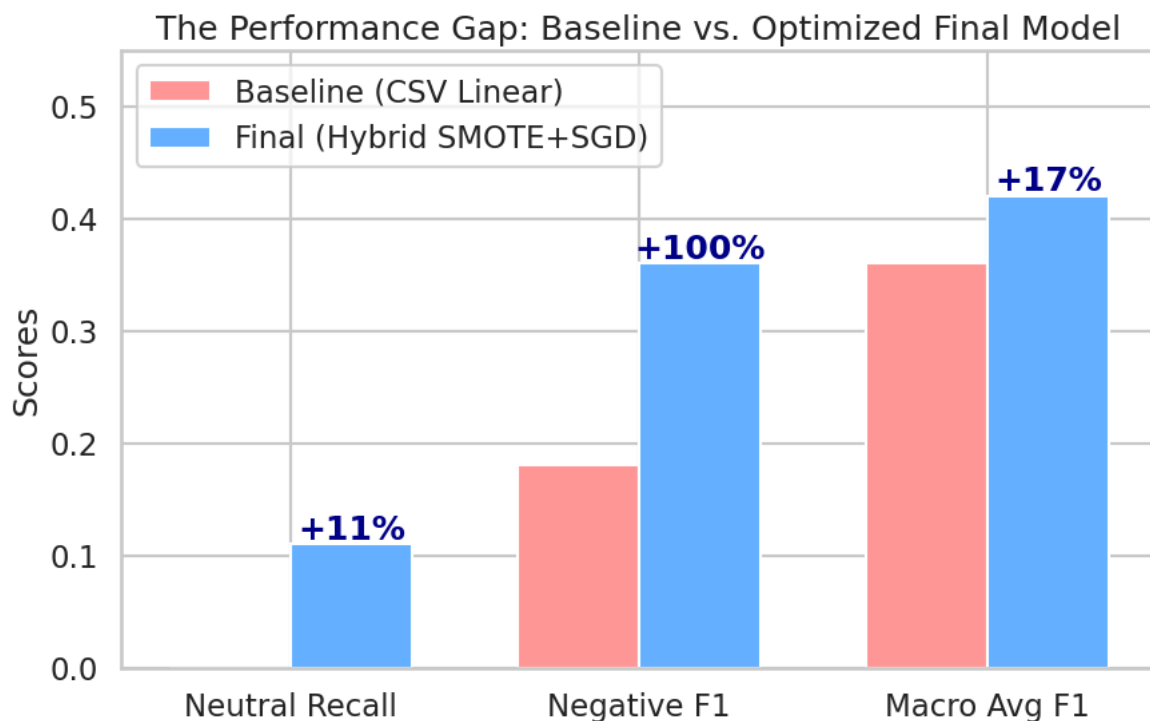
fig, ax = plt.subplots(figsize=(10, 6))
rects1 = ax.bar(x - width/2, baseline, width, label='Baseline (CSV Linear)', color='#ff9999')
rects2 = ax.bar(x + width/2, hybrid_final, width, label='Final (Hybrid SMOTE+SGD)', color='#66b3ff')

ax.set_ylabel('Scores')
ax.set_title('The Performance Gap: Baseline vs. Optimized Final Model')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()

```

```
# Add improvement percentage labels
for i in range(len(labels)):
    improvement = ((hybrid_final[i] - baseline[i]) / (baseline[i] if baseline[i] != 0 else 1)) * 100
    ax.annotate(f'+{improvement:.0f}%',
               xy=(x[i] + width/2, hybrid_final[i]),
               xytext=(0, 3), textcoords="offset points",
               ha='center', fontweight='bold', color='darkblue')

plt.ylim(0, 0.55)
plt.show()
```



##Parallel Coordinates Plot: SVM Evolution

```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates

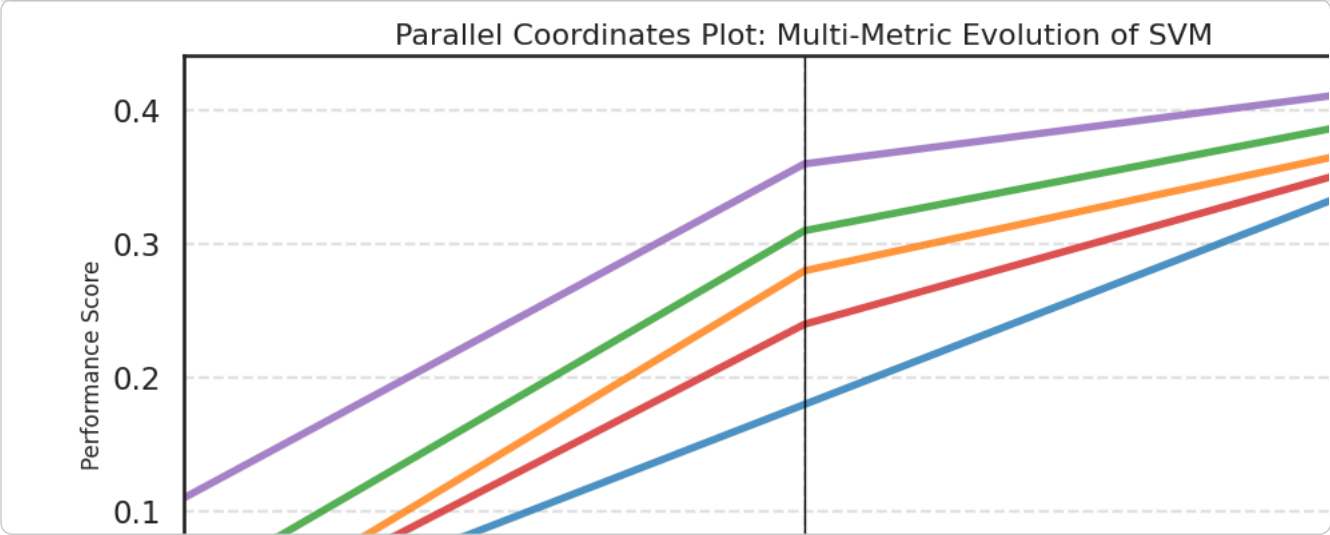
# 1. Load the evolution data
df = pd.read_csv('SVM_Model_Evolution_Comparison.csv')

# 2. Select metrics for comparison
# We focus on the key performance indicators
plot_df = df[['Method', 'Neutral_Recall', 'Negative_F1', 'Macro_Avg_F1']]

# 3. Create the plot
plt.figure(figsize=(12, 6))
parallel_coordinates(plot_df, 'Method',
                    color=('1f77b4', 'ff7f0e', '2ca02c', 'd62728', '9467bd'),
                    alpha=0.8, linewidth=4)

# 4. Styling and Labels
plt.title('Parallel Coordinates Plot: Multi-Metric Evolution of SVM', fontsize=16)
plt.ylabel('Performance Score', fontsize=12)
plt.xlabel('Metric Categories', fontsize=12)
plt.grid(True, axis='both', linestyle='--', alpha=0.5)
plt.legend(title='SVM Version', loc='upper right', bbox_to_anchor=(1.3, 1))

# 5. Save the output
plt.savefig('SVM_Parallel_Coordinates_Plot.png', dpi=300, bbox_inches='tight')
```



```
import pandas as pd

final_comp = {
    'Model Type': ['Random Forest', 'CSV Linear (SVM)', 'CSV_COST (SVM)', 'SMOTE (SVM)', 'Hybrid (SMOTE+SGD)'],
    'Neutral_Recall': [0.42, 0.00, 0.00, 0.04, 0.11],
    'Macro_F1': [0.61, 0.36, 0.38, 0.40, 0.42],
    'Scalability': ['Low (Sub-sampled)', 'Low', 'Low', 'Medium', 'High (Full Data)']
}

pd.DataFrame(final_comp).to_csv('Full_Model_Comparison_Report.csv', index=False)
print("Final comparison report saved!")
```

Final comparison report saved!

Start coding or [generate](#) with AI.

```
from google.colab import files

# List all the file names you saved in the previous step
plot_files = [
    'SVM_Confusion_Matrix.png',
    'SVM_Metrics_Comparison.png',
    'SVM_Precision_Recall_Curve.png',
    'SVM_Decision_Boundary.png'
]

for file_path in plot_files:
    try:
        files.download(file_path)
    except:
        print(f"File {file_path} not found. Check if you ran the save code.")
```

Start coding or [generate](#) with AI.

```
import pandas as pd
from google.colab import files

# 1. Create the Performance Metrics Table
metrics_data = {
```